Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
дисциплины
«Основы нейронных сетей»
Вариант № 14

Выполнила:
Текеева Мадина Азрет-Алиевна
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Проверил:
Доцент департамента цифровых, робототехнических систем и электроники института перспективной инженерии Воронкин Р.А

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г.

Тема: Изучение архитектуры и принципов работы сверточных нейронных сетей (CNN) для задач компьютерного зрения.

Цель работы: освоение теоретических основ и приобретение практических навыков построения сверточных нейронных сетей.

Репозиторий: https://github.com/bickrosss/NN_lab3

Порядок выполнения работы:

Задание 1 Вместо обучения свёрточной сети с нуля используется fine-tuning – дообучение готовой архитектуры ResNet50, предварительно обученной на ImageNet.

1.1 Реализация средствами библиотеки Keras.

1.1.1 Подключение необходимых для работы библиотек и загрузка датасета.

Импортированы необходимые инструменты (TensorFlow, NumPy, Matplotlib) и выполнена загрузка датасета рукописных цифр MNIST. Это базовый этап для подготовки среды разработки и получения исходных данных для обучения.

```
# загрузка датасета MNIST
from tensorflow.keras.datasets import mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

Рисунок 1.1.1 – Результат загрузки датасета MNIST

```
# Подключение утилит для to_categorical
from tensorflow.keras import utils

# Подключение библиотеки для работы с массивами
import numpy as np

# Подключение библиотек для отрисовки изображений
import matplotlib.pyplot as plt

# Подключение библиотеки для генерации случайных чисел
import random

# Подключение класса для работы с изображением
from PIL import Image

# Вывод изображений в ноутбуке, а не в консоли или файле
%matplotlib inline
```

Рисунок 1.1.2 – Подключение библиотек

1.1.2 Визуализация изображений из датасета.

Для ознакомления с содержимым датасета выводится по одному случайному изображению из каждого класса (цифры от 0 до 9). Это позволяет убедиться в корректности загрузки данных и оценить характер исходных изображений.

Листинг 1.1.1 – Вывод изображения каждого класса

```
#  вывод изображений каждого класса для ознакомления с датасетом

fig, axs = plt.subplots(1, 10, figsize=(25,3)) # создаем полотно для 10
графиков с размером 25 на 3
for i in range(10):
    label_indexes = np.where(y_train == i)[0] # получаем список из
индексов положений класса i в y_train
    index = random.choice(label_indexes) # выбирает случайный индекс из
списка созданного выше
    img = x_train[index] # выбираем из x_train нужное положение
    axs[i].imshow(Image.fromarray(img), cmap='gray') # выводим изображение

plt.show()
```

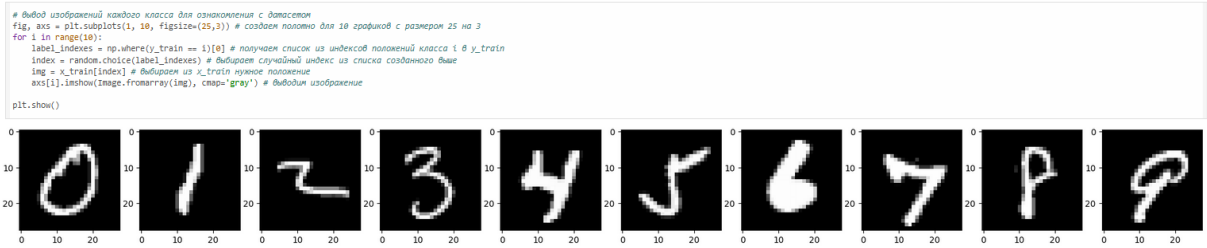


Рисунок 1.1.3 – Изображения из датасета MNIST

1.1.3 Предобработка данных.

Выполнено изменение формы тензоров (добавление канала цветности) и преобразование меток классов в формат One-Hot Encoding (OHE). Эти действия необходимы для приведения данных к виду, который принимает нейросеть.

```
# Подготовка данных под fine-tuning ResNet50
# ResNet50 ждёт на входе 3-канальные изображения. MNIST — одноканальный.
# Чтобы не раздувать numpy-массив, дублирование каналов и ресайз сделаем
# прямо ВНУТРИ модели (слоями Keras). Здесь — только reshape и OHE.

x_train = x_train.reshape(x_train.shape[0], 28, 28, 1).astype('float32')
x_test  = x_test.reshape(x_test.shape[0], 28, 28, 1).astype('float32')

y_train = utils.to_categorical(y_train, 10)
y_test  = utils.to_categorical(y_test, 10)

print('x_train:', x_train.shape)
print('x_test :', x_test.shape)
print('y_train:', y_train.shape)
print('y_test :', y_test.shape)
```

```
x_train: (60000, 28, 28, 1)
x_test : (10000, 28, 28, 1)
y_train: (60000, 10)
y_test : (10000, 10)
```

Рисунок 1.1.4 – Размерности выборок датасета

1.1.4 Конструирование архитектуры модели на базе ResNet50

Создана модель, включающая предобученную базу ResNet50 и кастомную «голову» (полносвязные слои, Dropout, BatchNormalization). Внутри модели добавлены слои масштабирования (Resizing) и конвертации в 3-канальное изображение, чтобы адаптировать MNIST под требования ResNet50.

Листинг 1.1.2 – Архитектура нейронной сети

```
#  Размер, до которого ресайзим картинки
# ResNet50 формально требует минимум 32x32, но при таком размере признаки
извлекаются плохо.
# Берём 64x64 - компромисс между качеством признаков и потреблением
памяти.
IMG_SIZE = 64

#  Входной тензор и предобработка прямо в модели
inputs = Input(shape=(28, 28, 1))                        # оригинал MNIST
x = Concatenate(axis=-1)([inputs, inputs, inputs])       # 1 канал -> 3
канала (повтор)
x = Resizing(IMG_SIZE, IMG_SIZE)(x)                      # 28x28 -> 64x64
x = Lambda(preprocess_input)(x)                          # нормализация под
ImageNet

#  Загружаем предобученную базу ResNet50
# include_top=False - без её родного классификатора на 1000 классов
ImageNet,
# свою голову на 10 классов поставим сами ниже.
base_model = ResNet50(
    weights='imagenet',
```

```
    include_top=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
)

# На 1-м этапе вся база заморожена - обучаться будет только голова
base_model.trainable = False

x = base_model(x, training=False)         # training=False важно для
BatchNorm в замороженной базе
x = GlobalAveragePooling2D()(x)           # свёртываем карту признаков в
вектор
x = BatchNormalization()(x)               # нормализация
x = Dense(128, activation='relu')(x)      # полносвязный слой
x = Dropout(0.3)(x)                       # регуляризация
outputs = Dense(10, activation='softmax')(x)

model = Model(inputs, outputs)
model.summary()
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applicatior
94765736/94765736 ━━━━━━━━━━━━━━━━━━━━ 6s 0us/step
Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 28, 28, 1)	0	-
concatenate (Concatenate)	(None, 28, 28, 3)	0	input_layer[0][0… input_layer[0][0… input_layer[0][0]
resizing (Resizing)	(None, 64, 64, 3)	0	concatenate[0][0]
lambda (Lambda)	(None, 64, 64, 3)	0	resizing[0][0]
resnet50 (Functional)	(None, 2, 2, 2048)	23,587,712	lambda[0][0]
global_average_poo… (GlobalAveragePool…	(None, 2048)	0	resnet50[0][0]
batch_normalization (BatchNormalizatio…	(None, 2048)	8,192	global_average_p…
dense (Dense)	(None, 128)	262,272	batch_normalizat…
dropout (Dropout)	(None, 128)	0	dense[0][0]
dense_1 (Dense)	(None, 10)	1,290	dropout[0][0]

Total params: 23,859,466 (91.02 MB)
Trainable params: 267,658 (1.02 MB)
Non-trainable params: 23,591,808 (90.00 MB)

Рисунок 1.1.5 – Структура созданной нейронной сети

1.1.5 Настройка двухэтапного обучения.

Описан процесс обучения в два этапа: сначала тренируется только новая «голова» при замороженной базе, затем размораживаются верхние слои ResNet50 для тонкой донастройки на низкой скорости обучения (learning rate). Это позволяет эффективно использовать предобученные веса и избежать их «поломки».

Листинг1.1.3 – Fine-tuning

```
#  ЭТАП 1: обучаем только голову, база заморожена
```

```
model.compile(
    loss='categorical_crossentropy',
    optimizer=Adam(learning_rate=1e-3),
    metrics=['accuracy'],
)

EPOCHS_STAGE1 = 5      # быстро адаптируем голову
EPOCHS_STAGE2 = 10     # затем тонко донастраиваем верхние блоки базы
# Итого 15 эпох - как в задании

print('=' * 60)
print('ЭТАП 1: обучение головы (база ResNet50 заморожена)')
print('=' * 60)
history1 = model.fit(
    x_train, y_train,
    batch_size=128,
    epochs=EPOCHS_STAGE1,
    validation_data=(x_test, y_test),
    verbose=1,
)

#  ЭТАП 2: размораживаем верхние блоки ResNet50
# Размораживать ВСЁ для маленького датасета как MNIST рискованно -
переобучимся.
# Поэтому размораживаем только верхнюю часть сети (примерно последние 30
слоёв).
base_model.trainable = True
for layer in base_model.layers[:-30]:
    layer.trainable = False

# Перекомпилируем с очень маленьким lr (1e-5), чтобы не «сломать»
предобученные веса
model.compile(
    loss='categorical_crossentropy',
    optimizer=Adam(learning_rate=1e-5),
    metrics=['accuracy'],
)

print()
print('=' * 60)
print('ЭТАП 2: fine-tuning верхних блоков ResNet50 (lr = 1e-5)')
print('=' * 60)
history2 = model.fit(
    x_train, y_train,
    batch_size=128,
    epochs=EPOCHS_STAGE2,
    validation_data=(x_test, y_test),
    verbose=1,
)

# Склеиваем истории двух этапов в одну
```

```
history = {
    'accuracy':     history1.history['accuracy']     +
history2.history['accuracy'],
    'val_accuracy': history1.history['val_accuracy'] +
history2.history['val_accuracy'],
    'loss':         history1.history['loss']         +
history2.history['loss'],
    'val_loss':     history1.history['val_loss']     +
history2.history['val_loss'],
}
```

```
============================================================
ЭТАП 1: обучение головы (база ResNet50 заморожена)
============================================================
Epoch 1/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 42s 61ms/step - accuracy: 0.9267 - loss: 0.2381 - val_accuracy: 0.9672 - val_loss: 0.1035
Epoch 2/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 16s 33ms/step - accuracy: 0.9652 - loss: 0.1056 - val_accuracy: 0.9713 - val_loss: 0.0921
Epoch 3/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 16s 34ms/step - accuracy: 0.9751 - loss: 0.0760 - val_accuracy: 0.9761 - val_loss: 0.0872
Epoch 4/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 17s 35ms/step - accuracy: 0.9787 - loss: 0.0648 - val_accuracy: 0.9756 - val_loss: 0.0946
Epoch 5/5
469/469 ━━━━━━━━━━━━━━━━━━━━ 17s 36ms/step - accuracy: 0.9814 - loss: 0.0561 - val_accuracy: 0.9753 - val_loss: 0.0900

============================================================
ЭТАП 2: fine-tuning верхних блоков ResNet50 (lr = 1e-5)
============================================================
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 59s 82ms/step - accuracy: 0.9316 - loss: 0.2330 - val_accuracy: 0.9708 - val_loss: 0.0939
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 23s 48ms/step - accuracy: 0.9761 - loss: 0.0727 - val_accuracy: 0.9772 - val_loss: 0.0715
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 23s 48ms/step - accuracy: 0.9866 - loss: 0.0399 - val_accuracy: 0.9798 - val_loss: 0.0637
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 23s 48ms/step - accuracy: 0.9916 - loss: 0.0256 - val_accuracy: 0.9820 - val_loss: 0.0601
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 23s 48ms/step - accuracy: 0.9942 - loss: 0.0171 - val_accuracy: 0.9819 - val_loss: 0.0562
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 23s 48ms/step - accuracy: 0.9962 - loss: 0.0117 - val_accuracy: 0.9831 - val_loss: 0.0567
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 23s 48ms/step - accuracy: 0.9971 - loss: 0.0094 - val_accuracy: 0.9839 - val_loss: 0.0539
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 23s 48ms/step - accuracy: 0.9979 - loss: 0.0069 - val_accuracy: 0.9845 - val_loss: 0.0531
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 23s 48ms/step - accuracy: 0.9986 - loss: 0.0051 - val_accuracy: 0.9857 - val_loss: 0.0524
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 23s 48ms/step - accuracy: 0.9987 - loss: 0.0048 - val_accuracy: 0.9849 - val_loss: 0.0567
```

Рисунок 1.1.6 – Процесс обучения модели

На первом этапе (замороженная база) модель достигла точности 98.14% на обучающей выборке за 5 эпох.

После разморозки верхних блоков точность на обучении достигла 99.87%, что почти является идеальным результатом. Однако на графике заметен характерный «скачок» потерь (loss) в момент начала fine-tuning (красная пунктирная линия). Это происходит из-за того, что веса предобученных слоев начинают подстраиваться под специфику MNIST, кратковременно нарушая стабильность сети перед выходом на новый максимум.

1.1.6 Построение графика обучения.

Для визуальной оценки процесса обучения строятся два графика: динамика доли верных ответов (accuracy) и динамика функции ошибки (loss) на обучающей и проверочной выборках в зависимости от номера эпохи.

Листинг 1.1.4 – Построение графика обучения

График обучения

```
plt.figure(figsize=(14, 5))

# График accuracy
plt.subplot(1, 2, 1)
plt.plot(history['accuracy'],     label='Обучающая выборка')
plt.plot(history['val_accuracy'], label='Проверочная выборка')
plt.axvline(x=EPOCHS_STAGE1 - 0.5, color='red', linestyle='--', alpha=0.6,
label='Старт fine-tuning')
plt.title('Доля верных ответов (accuracy)')
plt.xlabel('Эпоха')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# График loss
plt.subplot(1, 2, 2)
plt.plot(history['loss'],     label='Обучающая выборка')
plt.plot(history['val_loss'], label='Проверочная выборка')
plt.axvline(x=EPOCHS_STAGE1 - 0.5, color='red', linestyle='--', alpha=0.6,
label='Старт fine-tuning')
plt.title('Функция ошибки (loss)')
plt.xlabel('Эпоха')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

print(f"Итоговая точность на обучающей выборке:   {history['accuracy'][-
1]*100:.2f}%")
print(f"Итоговая точность на проверочной выборке:
{history['val_accuracy'][-1]*100:.2f}%")
```

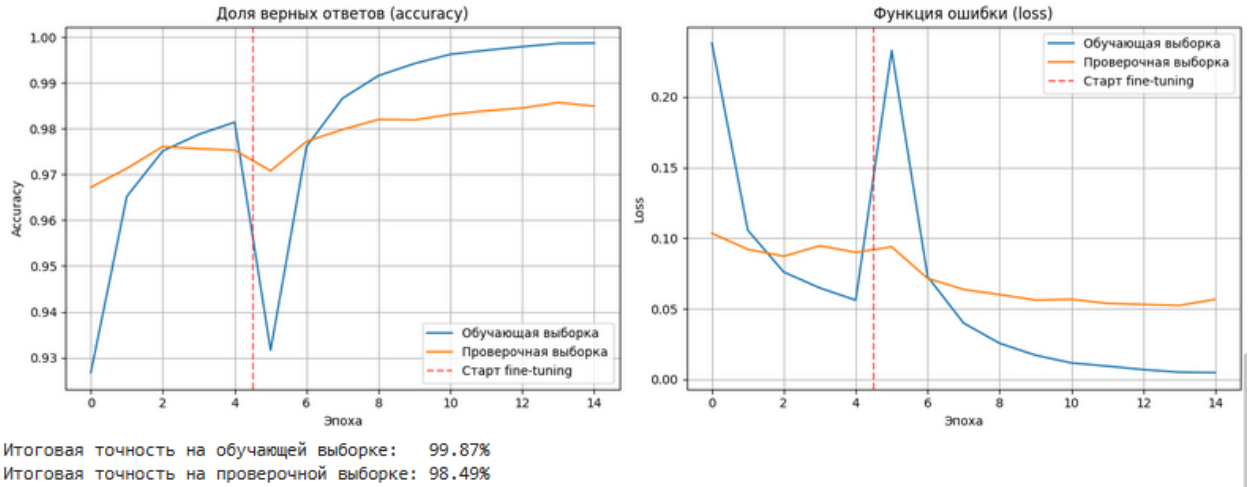


Рисунок 1.1.7 – График процесса обучения модели

На графике видно, что доля верных ответов на обоих наборах данных стабильно растёт и быстро выходит на плато около 99%. Кривые ошибки также демонстрируют успешное обучение: после резкого падения на первой эпохе значения функции потерь продолжают плавно снижаться, без расхождения между обучающей и проверочной выборками, что свидетельствует об отсутствии переобучения.

1.1.7 Построение матрицы ошибок.

Для более детального анализа качества классификации строится нормализованная матрица ошибок. Она показывает, какая доля изображений каждого класса была верно распознана моделью, а в каких случаях возникли ошибки. Также вычисляется общая точность работы модели на тестовой выборке.

Листинг 1.1.5 – Построение матрицы ошибок

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Получаем предсказания модели на тестовой выборке
y_pred = model.predict(x_test)

# Преобразуем ОНЕ в номера классов
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = np.argmax(y_test, axis=1)

# Строим матрицу ошибок (нормализованную)
cm = confusion_matrix(y_true_classes, y_pred_classes, normalize='true')
cm = np.round(cm, 3)
```

```
# Отображаем матрицу ошибок
fig, ax = plt.subplots(figsize=(10, 10))
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=np.arange(10))
disp.plot(ax=ax, cmap='Greens', colorbar=True)
plt.title('Матрица ошибок на тестовой выборке')
plt.xlabel('Предсказанный класс')
plt.ylabel('Истинный класс')
plt.show()

# Вывод общей точности
accuracy = np.mean(y_pred_classes == y_true_classes)
print(f'\nОбщая точность на тестовой выборке: {accuracy:.4f}
({accuracy*100:.2f}%)')
```

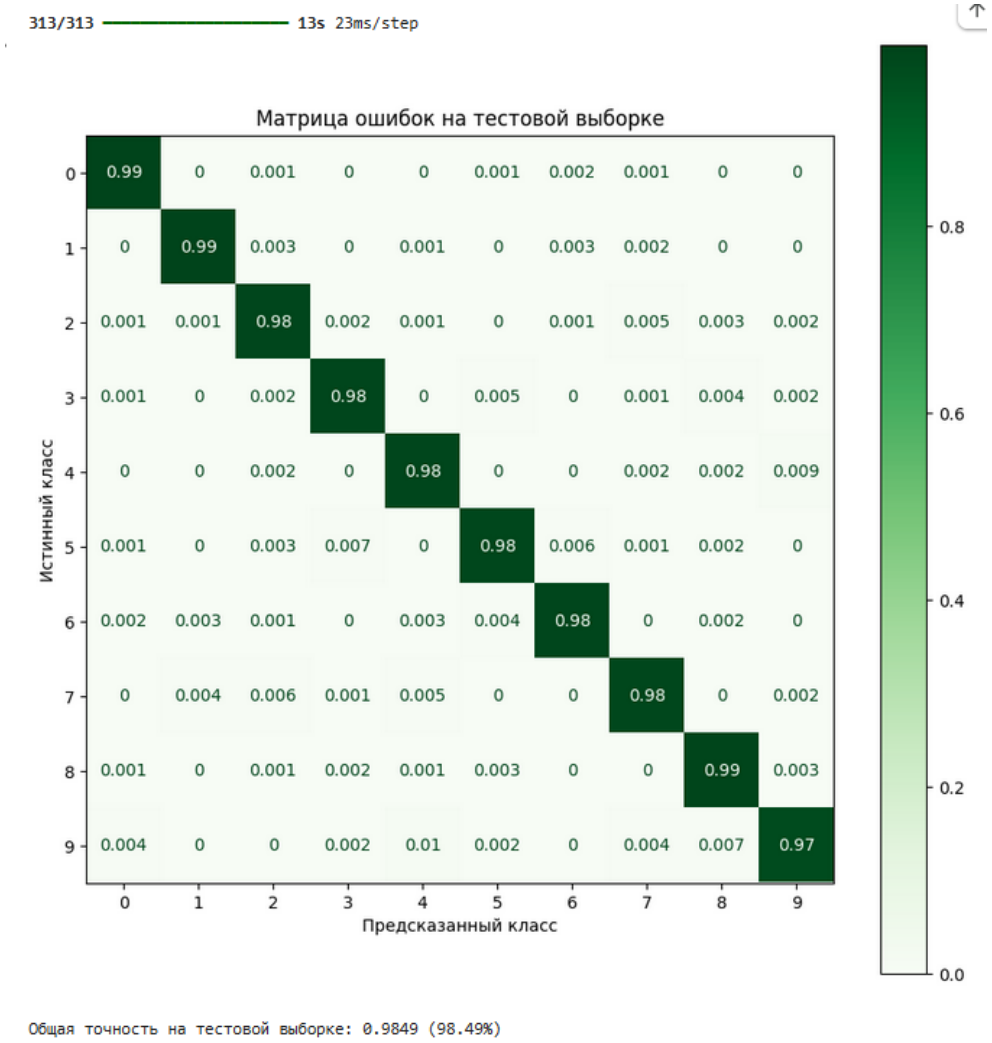


Рисунок 1.1.8 – Матрица ошибок на тестовой выборке

Анализ матрицы ошибок показывает, что итоговая точность составила 98.49%. Самая «проблемная» цифра – 9, которую модель в 0.7% случаев путает

с 4 и в 0.4% с 0. Это объясняется тем, что при низком разрешении (64x64) и специфическом почерке эти цифры имеют схожие контуры.

1.2 Реализация той же задачи распознавания рукописных цифр с использованием библиотеки PyTorch.

1.2.1 Подключение необходимых библиотек.

В отличие от Keras, в PyTorch модули импортируются по отдельности: torch для основных операций, torch.nn для слоёв нейронной сети, torch.optim для оптимизаторов, torch.utils.data для загрузчиков данных, torchvision для готовых датасетов и преобразований. Также определяется устройство, на котором будет производиться обучение (GPU при наличии, иначе CPU).

Загрузка библиотек

In [1]:

```
# PyTorch
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# Базовые библиотеки
import numpy as np
import matplotlib.pyplot as plt
import random

# Метрики
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Вывод изображений в ноутбуке
%matplotlib inline

# Определение устройства (GPU если доступен)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Используется устройство: {device}')
```

```
Используется устройство: cuda
```

Рисунок 1.2.1 – Подключение библиотек и определение устройства

1.2.2 Загрузка датасета MNIST.

Создан пайплайн преобразования изображений: изменение размера до 64x64, перевод в RGB (3 канала), преобразование в тензор и нормализация под параметры ImageNet. Это необходимо для корректной работы с предобученной моделью ResNet50.

```python
# Преобразования под ResNet50:
# 1. Ресайз 28x28 -> 64x64 (ResNet50 ждёт минимум 32x32; 64 — разумный компромисс).
# 2. Grayscale -> RGB (повтор единственного канала 3 раза).
# 3. ToTensor: [0..255] -> [0..1], порядок каналов HWC -> CHW.
# 4. Нормализация под ImageNet (те же mean/std, на которых училась ResNet50).
IMG_SIZE = 64

transform = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),            # 28x28 -> 64x64
    transforms.Grayscale(num_output_channels=3),        # 1 канал -> 3 канала (RGB)
    transforms.ToTensor(),                              # -> [0, 1], CHW
    transforms.Normalize(                               # нормализация под ImageNet
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

# Загрузка обучающей и проверочной выборок
train_dataset = datasets.MNIST(root='./data', train=True,  download=True, transform=transform)
test_dataset  = datasets.MNIST(root='./data', train=False, download=True, transform=transform)

print(f'Размер обучающей выборки: {len(train_dataset)}')
print(f'Размер тестовой выборки:  {len(test_dataset)}')
print(f'Форма одного изображения: {train_dataset[0][0].shape}')  # ожидаем (3, 64, 64)
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 18.0MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 484kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.47MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 14.7MB/s]Размер обучающей выборки: 60000
Размер тестовой выборки:  10000
Форма одного изображения: torch.Size([3, 64, 64])
```

Рисунок 1.2.2 – Результат загрузки датасета MNIST

1.2.3 Визуализация изображений из датасета.

Аналогично первому разделу, для ознакомления с содержимым датасета выводится по одному случайному изображению из каждого класса. Изображения извлекаются непосредственно из объекта train_dataset через атрибуты data и targets.

Листинг 1.2.1 – Вывод изображений каждого класса

```python
# Просмотр изображений каждого класса для ознакомления с датасетом.
# Берём из train_dataset.data - это исходные 28x28 ЧБ картинки (до
transform).
fig, axs = plt.subplots(1, 10, figsize=(25, 3))

all_labels = train_dataset.targets.numpy()
all_images = train_dataset.data.numpy()

for i in range(10):
    label_indexes = np.where(all_labels == i)[0]
    index = random.choice(label_indexes)
    img = all_images[index]
    axs[i].imshow(img, cmap='gray')
    axs[i].set_title(f'Класс: {i}')
    axs[i].axis('off')
```

```
plt.show()
```

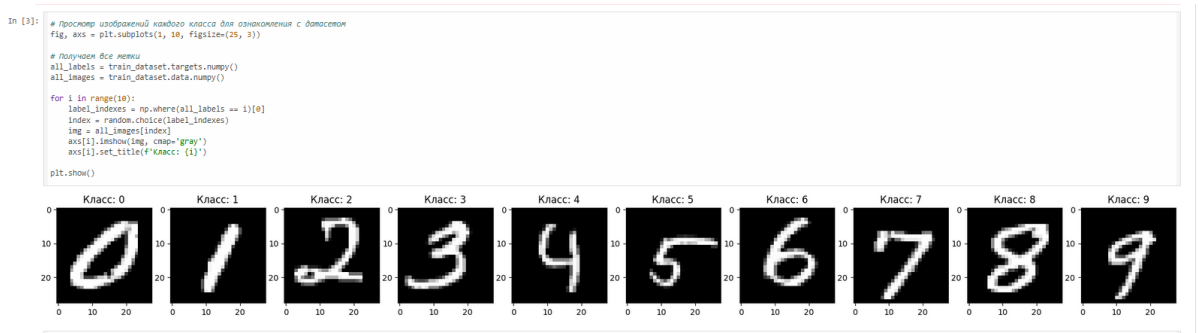


Рисунок 1.2.3 – Изображения из датасета MNIST

1.2.4 Создание загрузчиков данных.

Объекты DataLoader сформированы с размером батча 128. Они отвечают за эффективную подачу данных в модель порциями, перемешивание обучающей выборки и использование параллельных потоков процессора для ускорения загрузки..

```
# Создание DataLoader-ов с размером батча 128
BATCH_SIZE = 128

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,  num_workers=2)
test_loader  = DataLoader(test_dataset,  batch_size=BATCH_SIZE, shuffle=False, num_workers=2)

# Проверка размерностей
for images, labels in train_loader:
    print(f'Форма батча изображений: {images.shape}')   # ожидаем [128, 3, 64, 64]
    print(f'Форма батча меток:       {labels.shape}')   # [128]
    break
```

```
Форма батча изображений: torch.Size([128, 3, 64, 64])
Форма батча меток:       torch.Size([128])
```

Рисунок 1.2.4 – Размерности батча в DataLoader

1.2.5 Модификация архитектуры ResNet50.

Загружена модель ResNet50 с предобученными весами. На первом этапе все веса заморожены (requires_grad = False), а финальный полносвязный слой заменен на новую «голову», адаптированную под 10 классов цифр. Это позволяет обучать только новые слои, сохраняя накопленные знания базовой сети.

Листинг 1.2.2 – Класс модели CNN

```
# Загружаем предобученную ResNet50 (на ImageNet)
```

```
resnet = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)

#  ЭТАП 1: замораживаем ВСЁ
for param in resnet.parameters():
    param.requires_grad = False

#  Заменяем последний слой (голову) на свой
# Параметры новой головы по умолчанию имеют requires_grad=True - то, что
нам нужно.
num_features = resnet.fc.in_features  # 2048
resnet.fc = nn.Sequential(
    nn.BatchNorm1d(num_features),
    nn.Linear(num_features, 128),
    nn.ReLU(inplace=True),
    nn.Dropout(0.3),
    nn.Linear(128, 10),
)

model = resnet.to(device)

# Подсчёт параметров - на 1-м этапе обучаемых должно быть в десятки раз
меньше общего числа
total_params     = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
print(f'Всего параметров:     {total_params:,}')
print(f'Обучаемых (этап 1):   {trainable_params:,}')
```

```
Downloading: "https://download.pytorch.org/models/resnet50-11ad3fa6.pth" to /root/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth
100%|██████████| 97.8M/97.8M [00:00<00:00, 180MB/s]
Всего параметров:     23,775,690
Обучаемых (этап 1):   267,658
```

Рисунок 1.2.5 – Структура модели и количество параметров

1.2.6 Обучение модели.

Реализована универсальная функция train_model, которая выполняет стандартный цикл: прямой проход, расчет функции потерь (CrossEntropy), обратное распространение ошибки и обновление весов. Функция также собирает статистику точности и потерь для последующего анализа.

Листинг 1.2.3 – Цикл обучения

```
#  Функция обучения (один проход)
criterion = nn.CrossEntropyLoss()

def train_model(model, optimizer, n_epochs, history, epoch_offset=0):
    '''Обучает модель n_epochs эпох, дописывая метрики в history.'''
    for epoch in range(n_epochs):
        #  Фаза обучения
```

```
        model.train()
        train_loss_sum = 0.0
        train_correct  = 0
        train_total    = 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            train_loss_sum += loss.item() * images.size(0)
            _, predicted = outputs.max(1)
            train_total   += labels.size(0)
            train_correct += predicted.eq(labels).sum().item()

        train_loss = train_loss_sum / train_total
        train_acc  = train_correct  / train_total

        #  Фаза валидации
        model.eval()
        val_loss_sum = 0.0
        val_correct  = 0
        val_total    = 0

        with torch.no_grad():
            for images, labels in test_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                loss = criterion(outputs, labels)

                val_loss_sum += loss.item() * images.size(0)
                _, predicted = outputs.max(1)
                val_total   += labels.size(0)
                val_correct += predicted.eq(labels).sum().item()

        val_loss = val_loss_sum / val_total
        val_acc  = val_correct  / val_total

        history['train_loss'].append(train_loss)
        history['train_acc' ].append(train_acc)
        history['val_loss'  ].append(val_loss)
        history['val_acc'   ].append(val_acc)

        global_epoch = epoch + 1 + epoch_offset
        print(f'Эпоха {global_epoch:2d} | '
              f'train_loss: {train_loss:.4f} | train_acc: {train_acc:.4f}
| '
              f'val_loss: {val_loss:.4f} | val_acc: {val_acc:.4f}')
```

```
# Общая история на оба этапа
history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc':
[]}
EPOCHS_STAGE1 = 5
EPOCHS_STAGE2 = 10

#  ЭТАП 1: обучаем только голову, lr = 1e-3
# В optimizer передаём только параметры с requires_grad=True (т.е. новую
голову).
optimizer = optim.Adam(filter(lambda p: p.requires_grad,
model.parameters()), lr=1e-3)

print('ЭТАП 1: обучение головы (база ResNet50 заморожена)')
train_model(model, optimizer, EPOCHS_STAGE1, history, epoch_offset=0)

#  ЭТАП 2: размораживаем layer4 (последний residual-блок ResNet50), lr =
1e-5
# Размораживать всю ResNet50 для маленького датасета как MNIST рискованно
- переобучимся.
# Поэтому размораживаем только верхушку.
for param in model.layer4.parameters():
    param.requires_grad = True

trainable_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
print(f'\nОбучаемых параметров после разморозки layer4:
{trainable_params:,}')

# Очень маленький lr, чтобы не «сломать» предобученные веса
optimizer = optim.Adam(filter(lambda p: p.requires_grad,
model.parameters()), lr=1e-5)

print()
print('ЭТАП 2: fine-tuning layer4 (lr = 1e-5)')
train_model(model, optimizer, EPOCHS_STAGE2, history,
epoch_offset=EPOCHS_STAGE1)

print('\nОбучение завершено!')
```

```
ЭТАП 1: обучение головы (база ResNet50 заморожена)
Эпоха  1 | train_loss: 0.6818 | train_acc: 0.7795 | val_loss: 0.4932 | val_acc: 0.8355
Эпоха  2 | train_loss: 0.4950 | train_acc: 0.8369 | val_loss: 0.4441 | val_acc: 0.8531
Эпоха  3 | train_loss: 0.4630 | train_acc: 0.8458 | val_loss: 0.4543 | val_acc: 0.8515
Эпоха  4 | train_loss: 0.4365 | train_acc: 0.8535 | val_loss: 0.4214 | val_acc: 0.8613
Эпоха  5 | train_loss: 0.4200 | train_acc: 0.8598 | val_loss: 0.4057 | val_acc: 0.8654

Обучаемых параметров после разморозки layer4: 15,232,394

ЭТАП 2: fine-tuning layer4 (lr = 1e-5)
Эпоха  6 | train_loss: 0.3053 | train_acc: 0.8979 | val_loss: 0.3094 | val_acc: 0.9054
Эпоха  7 | train_loss: 0.2193 | train_acc: 0.9274 | val_loss: 0.2377 | val_acc: 0.9243
Эпоха  8 | train_loss: 0.1798 | train_acc: 0.9391 | val_loss: 0.2191 | val_acc: 0.9348
Эпоха  9 | train_loss: 0.1481 | train_acc: 0.9502 | val_loss: 0.2168 | val_acc: 0.9376
Эпоха 10 | train_loss: 0.1276 | train_acc: 0.9576 | val_loss: 0.1851 | val_acc: 0.9467
Эпоха 11 | train_loss: 0.1073 | train_acc: 0.9650 | val_loss: 0.1753 | val_acc: 0.9506
Эпоха 12 | train_loss: 0.0926 | train_acc: 0.9696 | val_loss: 0.1806 | val_acc: 0.9508
Эпоха 13 | train_loss: 0.0823 | train_acc: 0.9726 | val_loss: 0.1630 | val_acc: 0.9540
Эпоха 14 | train_loss: 0.0711 | train_acc: 0.9768 | val_loss: 0.1498 | val_acc: 0.9580
Эпоха 15 | train_loss: 0.0603 | train_acc: 0.9809 | val_loss: 0.1439 | val_acc: 0.9600

Обучение завершено!
```

Рисунок 1.2.6 – Результат обучения модели по эпохам

В логах видно, что на первом этапе модель PyTorch прогрессировала медленнее, достигнув 85.98% на обучении и 86.54% на валидации к 5-й эпохе. Это может быть связано с тем, что стандартная «голова» (классификатор) в этой реализации требовала больше времени на подстройку под выходы ResNet50.

После разморозки четвертого блока кривые точности и потерь после 5-й эпохи стали намного круче, что подтверждает: для этой реализации PyTorch базовых весов ImageNet было недостаточно, и дообучение внутренних слоев стало решающим фактором.

1.2.7 Построение графика обучения.

Поскольку история обучения сохранялась в обычный словарь Python, графики строятся аналогично Keras-версии. Выводятся две диаграммы: динамика accuracy и динамика loss на обучающей и проверочной выборках.

Листинг 1.2.4 – Построение графика обучения

```
plt.figure(figsize=(14, 5))

# Граница между этапом 1 и этапом 2 (для красной пунктирной линии)
stage_boundary = EPOCHS_STAGE1 - 0.5

# График accuracy
plt.subplot(1, 2, 1)
```

```
plt.plot(history['train_acc'], label='Обучающая выборка')
plt.plot(history['val_acc'],   label='Проверочная выборка')
plt.axvline(x=stage_boundary, color='red', linestyle='--', alpha=0.6,
label='Старт fine-tuning')
plt.title('Доля верных ответов (accuracy)')
plt.xlabel('Эпоха')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# График loss
plt.subplot(1, 2, 2)
plt.plot(history['train_loss'], label='Обучающая выборка')
plt.plot(history['val_loss'],   label='Проверочная выборка')
plt.axvline(x=stage_boundary, color='red', linestyle='--', alpha=0.6,
label='Старт fine-tuning')
plt.title('Функция ошибки (loss)')
plt.xlabel('Эпоха')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

print(f'Лучшая точность на проверочной выборке:
{max(history["val_acc"])*100:.2f}%')
print(f'Итоговая точность на проверочной выборке: {history["val_acc"][-
1]*100:.2f}%')
```

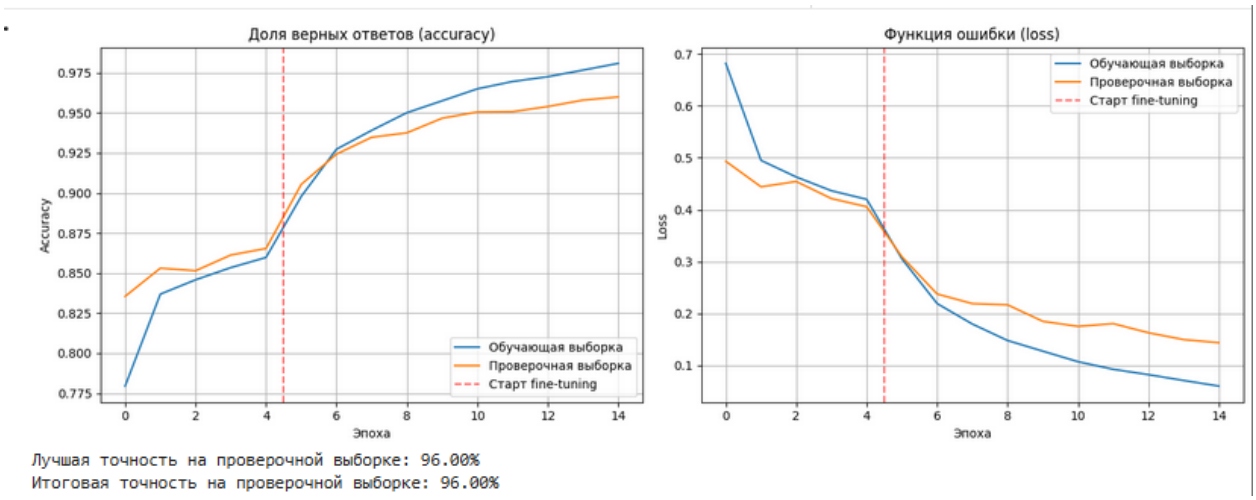


Рисунок 1.2.7 – График процесса обучения модели

1.2.8 Построение матрицы ошибок.

Для оценки качества классификации по каждому классу строится нормализованная матрица ошибок. Предсказания модели на тестовой выборке

собираются в списки, которые затем передаются в функцию confusion_matrix из библиотеки scikit-learn.

Листинг 1.2.5 – Построение матрицы ошибок

```
# Получение всех предсказаний на тестовой выборке
model.eval()
all_preds = []
all_labels = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        outputs = model(images)
        _, predicted = outputs.max(1)
        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.numpy())

all_preds = np.array(all_preds)
all_labels = np.array(all_labels)

# Построение нормализованной матрицы ошибок
cm = confusion_matrix(all_labels, all_preds, normalize='true')
cm = np.round(cm, 3)

fig, ax = plt.subplots(figsize=(10, 10))
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=np.arange(10))
disp.plot(ax=ax, cmap='Blues', colorbar=True)
plt.title('Матрица ошибок на тестовой выборке')
plt.xlabel('Предсказанный класс')
plt.ylabel('Истинный класс')
plt.show()

# Общая точность
accuracy = (all_preds == all_labels).mean()
print(f'\nОбщая точность на тестовой выборке: {accuracy:.4f}
({accuracy*100:.2f}%)')
```

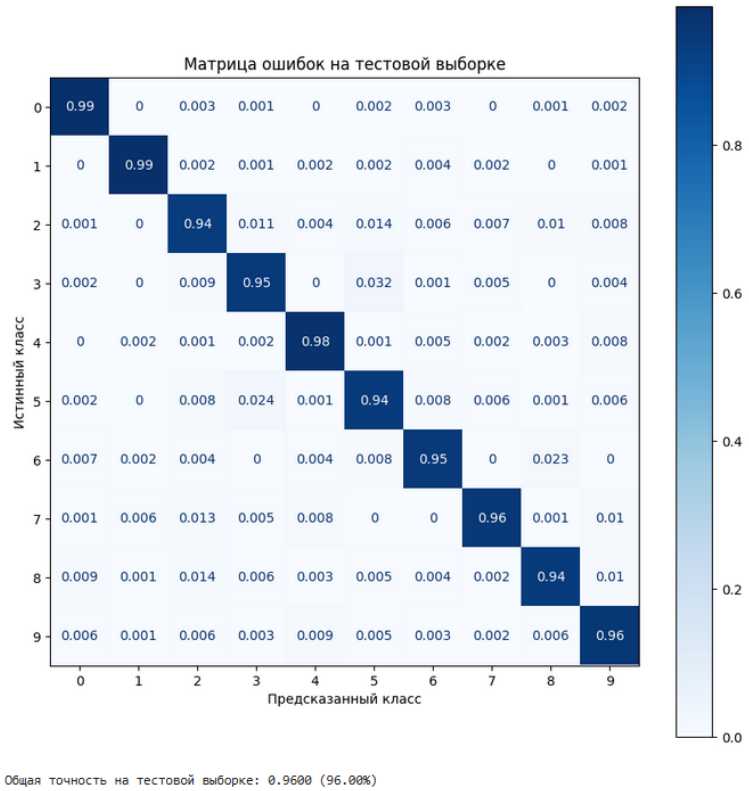


Рисунок 1.2.8 – Матрица ошибок на тестовой выборке

Сравнительный вывод: модель TensorFlow достигла стабильно высоких результатов гораздо раньше – фактически уже к 10-й эпохе суммарно. Модель PyTorch продолжала активно улучшаться вплоть до 15-й эпохи, что говорит о потенциале для дальнейшего обучения.

В обеих моделях точность на обучающей выборке выше, чем на проверочной (99.87% против 98.49% в TF и 98.09% против 96.00% в PyTorch). Это естественный процесс, но в модели TensorFlow разрыв меньше, что свидетельствует о чуть лучшей обобщающей способности данной конфигурации.

Задание 2. Решить задачу бинарной классификации изображений пассажиров автобуса по двум классам: «Входящий» и «Выходящий».

Цель – добиться точности работы модели выше 90% на проверочной выборке. Изображения представляют собой кадры с камер видеонаблюдения, на которых запечатлены пассажиры в полный рост в момент входа в автобус или выхода из него.

2.1 Подключение необходимых библиотек.

Для решения задачи импортируются модули для работы с изображениями (PIL, tensorflow.keras.preprocessing.image), массивами (numpy), графиками (matplotlib), файловой системой (os), генерации случайных чисел (random) и загрузки данных (gdown). Из библиотеки scikit-learn подключаются функции train_test_split для стратифицированного разбиения выборки, compute_class_weight для автоматического расчёта весов классов и confusion_matrix с classification_report для оценки качества модели. Из tensorflow.keras импортируются необходимые слои сети (Conv2D, MaxPooling2D, Dense, Dropout, BatchNormalization, LeakyReLU, GlobalAveragePooling2D), генератор аугментированных изображений ImageDataGenerator, колбэки EarlyStopping и ReduceLROnPlateau, а также оптимизатор Adam.

```
# Методы для отрисовки изображений
from PIL import Image

# Для отрисовки графиков
import matplotlib.pyplot as plt

# Для генерации случайных чисел
import random

# Библиотека работы с массивами
import numpy as np

# Для работы с файлами
import os

# Импортируем модуль для загрузки данных
import gdown

# Для разделения выборок
from sklearn.model_selection import train_test_split

# Для автоматического расчёта весов классов
from sklearn.utils.class_weight import compute_class_weight

# Матрица ошибок и отчёт классификатора
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report

# Генератор аугментированных изображений
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Для создания сети — теперь Model API (functional), а не Sequential
from tensorflow.keras.models import Model

# Для создания слоёв (для головы модели)
from tensorflow.keras.layers import (Input, Dense, Dropout, BatchNormalization,
                                     GlobalAveragePooling2D)

# Предобученная модель EfficientNetB0 + её собственный preprocess_input
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.applications.efficientnet import preprocess_input

# Для работы с изображениями
from tensorflow.keras.preprocessing import image

# Колбеки
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

# Преобразование меток в One-Hot
from tensorflow.keras.utils import to_categorical
```

Рисунок 2.1– Подключение библиотек

2.2 Загрузка датасета.

Архив с изображениями загружается с облачного хранилища при помощи библиотеки gdown и распаковывается в директорию /content/bus. После распаковки выводится список папок-категорий, который содержит две директории, соответствующие классам задачи: «Входящий» и «Выходящий».

```
# Загрузка датасета (.zip-архив)
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/14/bus.zip', None, quiet=True)

# Распаковка архива
!unzip -q "bus.zip" -d /content/bus

# Папка с папками картинок, рассортированных по категориям
IMAGE_PATH = '/content/bus'
os.listdir(IMAGE_PATH)
```

```
['Входящий', 'Выходящий']
```

Рисунок 2.2 – Загрузка и распаковка датасета bus.zip

2.3 Определение списка классов.

Список классов формируется автоматически на основе названий папок в директории датасета и сортируется по алфавиту, что обеспечивает воспроизводимое сопоставление меток. В результате имеем два класса: «Входящий» (индекс 0) и «Выходящий» (индекс 1).

```
# Определение списка имён классов
CLASS_LIST = sorted(os.listdir(IMAGE_PATH))

# Определение количества классов
CLASS_COUNT = len(CLASS_LIST)

print(f'Количество классов: {CLASS_COUNT}, метки классов: {CLASS_LIST}')
```

```
Количество классов: 2, метки классов: ['Входящий', 'Выходящий']
```

Рисунок 2.3 – Список классов и их количество

2.4 Гиперпараметры.

Установлены ключевые настройки обучения: доли разделения выборок, целевой размер изображений (96x192), параметры аугментации (повороты, сдвиги) и количество эпох для двух стадий обучения. Это позволяет гибко управлять процессом тренировки модели.

Гиперпараметры

```
# Параметры разделения выборок
TEST_SPLIT          = 0.1                   # Доля тестовых данных в общем наборе
VAL_SPLIT           = 0.2                   # Доля проверочной выборки в обучающем наборе

# Параметры изображения
# Соотношение 1:2 (фигура человека) сохраняем, но размер увеличен:
# EfficientNetB0 училась на ImageNet (224x224), и слишком маленькие картинки
# не дают раскрыть её потенциал. 96x192 — компромисс между качеством и скоростью.
IMG_WIDTH           = 96                    # Ширина изображения для нейросети
IMG_HEIGHT          = 192                   # Высота изображения для нейросети
IMG_CHANNELS        = 3                     # RGB

# Параметры аугментации
ROTATION_RANGE      = 15                    # Пределы поворота в градусах
WIDTH_SHIFT_RANGE   = 0.1                   # Пределы сдвига по горизонтали
HEIGHT_SHIFT_RANGE  = 0.1                   # Пределы сдвига по вертикали
ZOOM_RANGE          = 0.1                   # Пределы увеличения/уменьшения
BRIGHTNESS_RANGE    = (0.7, 1.3)            # Пределы изменения яркости
HORIZONTAL_FLIP     = False                 # Без флипа: направление движения важно

# Параметры обучения
EPOCHS_STAGE1       = 10                    # Этап 1: только голова
EPOCHS_STAGE2       = 30                    # Этап 2: fine-tuning (EarlyStopping остановит раньше)
BATCH_SIZE          = 32                    # Размер батча
LR_STAGE1           = 1e-3                  # Шаг для головы
LR_STAGE2           = 1e-5                  # Очень маленький шаг для fine-tuning
```

Рисунок 2.4 – Заданные гиперпараметры

2.5 Загрузка изображений.

Исходные файлы загружены и преобразованы в числовые массивы NumPy формата float32. Получено общее количество данных (9081 изображение) и проверена форма результирующего тензора.

Загрузка изображений

+ Код + Текст

```
images = []
labels = []

for i, class_name in enumerate(CLASS_LIST):
    class_path = os.path.join(IMAGE_PATH, class_name)
    for img_name in os.listdir(class_path):
        img = image.load_img(os.path.join(class_path, img_name),
                             target_size=(IMG_HEIGHT, IMG_WIDTH))
        images.append(image.img_to_array(img))
        labels.append(i)

images = np.array(images, dtype='float32')
labels = np.array(labels)

print(f'Всего изображений: {len(images)}')
print(f'Форма массива: {images.shape}')
print(f'Распределение классов: {np.bincount(labels)}')
```

```
Всего изображений: 9081
Форма массива: (9081, 192, 96, 3)
Распределение классов: [6485 2596]
```

Рисунок 2.5 – Загрузка изображений и статистика по выборке

2.6 Просмотр примеров изображений.

Для визуального ознакомления с содержимым датасета выводится по одному изображению из каждого класса. Это позволяет убедиться в корректности загрузки и оценить характер исходных кадров: качество, освещение, ракурс, типичные позы пассажиров.

```
# Просмотр по одному изображению из каждого класса
fig, axs = plt.subplots(1, CLASS_COUNT, figsize=(8, 5))
for i in range(CLASS_COUNT):
    idx = np.where(labels == i)[0][0]
    axs[i].imshow(images[idx].astype('uint8'))
    axs[i].set_title(CLASS_LIST[i])
    axs[i].axis('off')
plt.show()
```

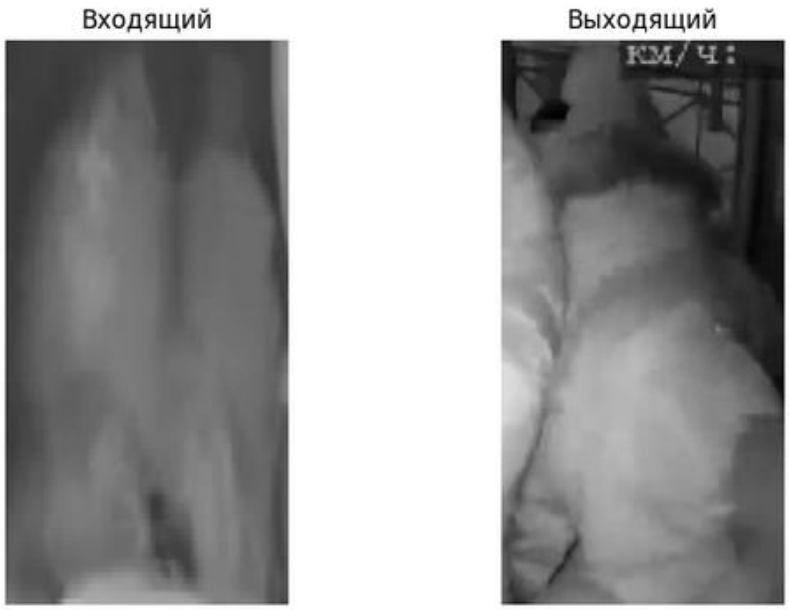


Рисунок 2.6 – Примеры изображений каждого класса

2.7 Разделение на train / val / test со стратификацией.

Весь массив данных разделен на обучающую (train), проверочную (val) и тестовую (test) выборки с применением стратификации. Метки переведены в формат One-Hot Encoding для корректной работы функции потерь.

В результате получены: обучающий набор – 6537 изображений, проверочный – 1635 изображений, тестовый – 909 изображений.

Разделение на train / val / test со стратификацией

+ Код + Текст

```python
# Сначала отделяем тестовую выборку (10% от всех данных)
x_train_full, x_test, y_train_full, y_test = train_test_split(
    images, labels,
    test_size=TEST_SPLIT,
    random_state=42,
    stratify=labels
)

# Затем делим оставшиеся данные на обучающую и проверочную (20% от обучающего набора)
x_train, x_val, y_train_raw, y_val_raw = train_test_split(
    x_train_full, y_train_full,
    test_size=VAL_SPLIT,
    random_state=42,
    stratify=y_train_full
)

# Преобразуем метки в One-Hot Encoding
y_train = to_categorical(y_train_raw, num_classes=CLASS_COUNT)
y_val = to_categorical(y_val_raw, num_classes=CLASS_COUNT)
y_test_ohe = to_categorical(y_test, num_classes=CLASS_COUNT)

print(f'Обучающий набор:    {x_train.shape}, метки: {y_train.shape}')
print(f'Проверочный набор:  {x_val.shape}, метки: {y_val.shape}')
print(f'Тестовый набор:     {x_test.shape}, метки: {y_test_ohe.shape}')
```

```
Обучающий набор:    (6537, 192, 96, 3), метки: (6537, 2)
Проверочный набор:  (1635, 192, 96, 3), метки: (1635, 2)
Тестовый набор:     (909, 192, 96, 3), метки: (909, 2)
```

Рисунок 2.7 – Разделение выборки на train, val и test

2.8 Проверка распределения классов.

Для проверки корректности стратификации выводится распределение классов в каждой из трёх выборок. Во всех трёх наборах сохраняется исходное соотношение: примерно 71.4% объектов класса «Входящий» и 28.6% объектов класса «Выходящий», что подтверждает успешную работу стратифицированного разбиения.

```python
# Проверка распределения классов в каждой выборке
def print_balance(y_data, set_name):
    if len(y_data.shape) > 1:
        y_data = np.argmax(y_data, axis=1)
    unique, counts = np.unique(y_data, return_counts=True)
    print(f'Распределение в {set_name}:')
    for i, count in enumerate(counts):
        print(f'  Класс {CLASS_LIST[i]}: {count} ({count/len(y_data)*100:.1f}%)')

print_balance(y_train, 'обучающем наборе')
print_balance(y_val, 'проверочном наборе')
print_balance(y_test_ohe, 'тестовом наборе')
```

```
Распределение в обучающем наборе:
  Класс Входящий: 4668 (71.4%)
  Класс Выходящий: 1869 (28.6%)
Распределение в проверочном наборе:
  Класс Входящий: 1168 (71.4%)
  Класс Выходящий: 467 (28.6%)
Распределение в тестовом наборе:
  Класс Входящий: 649 (71.4%)
  Класс Выходящий: 260 (28.6%)
```

Рисунок 2.8 – Распределение классов в обучающем, проверочном и тестовом наборах

2.9 Расчёт весов классов.

Для компенсации дисбаланса классов рассчитываются их веса при помощи функции compute_class_weight из scikit-learn в режиме «balanced». Этот режим присваивает каждому классу вес, обратно пропорциональный его частоте: классу «Входящий» – 0.700, классу «Выходящий» – 1.749. Полученные веса передаются в метод fit модели через параметр class_weight, что заставляет функцию ошибки сильнее реагировать на ошибки классификации миноритарного класса.

```python
# Автоматический расчёт весов классов для компенсации дисбаланса
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train_raw),
    y=y_train_raw
)
class_weight_dict = dict(enumerate(class_weights))
print(f'Веса классов: {class_weight_dict}')
```

```
Веса классов: {0: np.float64(0.7001928020565553), 1: np.float64(1.7487961476725522)}
```

Рисунок 2.9 – Расчёт весов классов

2.10 Аугментация изображений.

Созданы объекты ImageDataGenerator с применением специфической функции предобработки preprocess_input для EfficientNet. Для обучающей выборки включена динамическая аугментация (случайные повороты и изменения яркости), что помогает модели лучше обобщать данные и защищает от переобучения.

```
train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,    # вместо rescale
    rotation_range=ROTATION_RANGE,
    width_shift_range=WIDTH_SHIFT_RANGE,
    height_shift_range=HEIGHT_SHIFT_RANGE,
    zoom_range=ZOOM_RANGE,
    brightness_range=BRIGHTNESS_RANGE,
    horizontal_flip=HORIZONTAL_FLIP,
)

# Для проверочной и тестовой выборок — только preprocess_input (без аугментации)
val_datagen  = ImageDataGenerator(preprocessing_function=preprocess_input)
test_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)

# Создание генераторов
train_generator      = train_datagen.flow(x_train, y_train,     batch_size=BATCH_SIZE, shuffle=True)
validation_generator = val_datagen.flow(x_val,   y_val,         batch_size=BATCH_SIZE, shuffle=False)
test_generator       = test_datagen.flow(x_test, y_test_ohe,    batch_size=BATCH_SIZE, shuffle=False)
```

Рисунок 2.10 – Создание генераторов для аугментации и нормализации

2.11 Проверка работы аугментации.

Реализована функция обратного преобразования пикселей из диапазона [-1, 1] в [0, 1] для корректного отображения и выведены примеры изображений с наложенными метками классов «Входящий» и «Выходящий». Это позволяет убедиться, что генератор правильно применяет трансформации и сохраняет соответствие между картинкой и её аннотацией.

```
# Проверка работы аугментации.
# Данные после preprocess_input лежат в [-1, 1], для отображения возвращаем в [0, 1].
def show_batch(batch, img_range=range(10), figsize=(20, 8), columns=5):
    num_images = len(img_range)
    rows = (num_images + columns - 1) // columns
    fig, axes = plt.subplots(rows, columns, figsize=figsize)
    axes = axes.flatten()
    for i, img_idx in enumerate(img_range):
        img = batch[0][img_idx]
        # обратное преобразование для отображения: [-1, 1] -> [0, 1]
        img_show = np.clip((img + 1.0) / 2.0, 0, 1)
        axes[i].imshow(img_show)
        class_label_idx = np.argmax(batch[1][img_idx])
        axes[i].set_title(CLASS_LIST[class_label_idx])
        axes[i].axis('off')
    for j in range(i + 1, len(axes)):
        axes[j].axis('off')
    plt.tight_layout()
    plt.show()

example_batch = next(train_generator)
show_batch(example_batch, img_range=range(10))
```

Рисунок 2.11 – Код функции show_batc

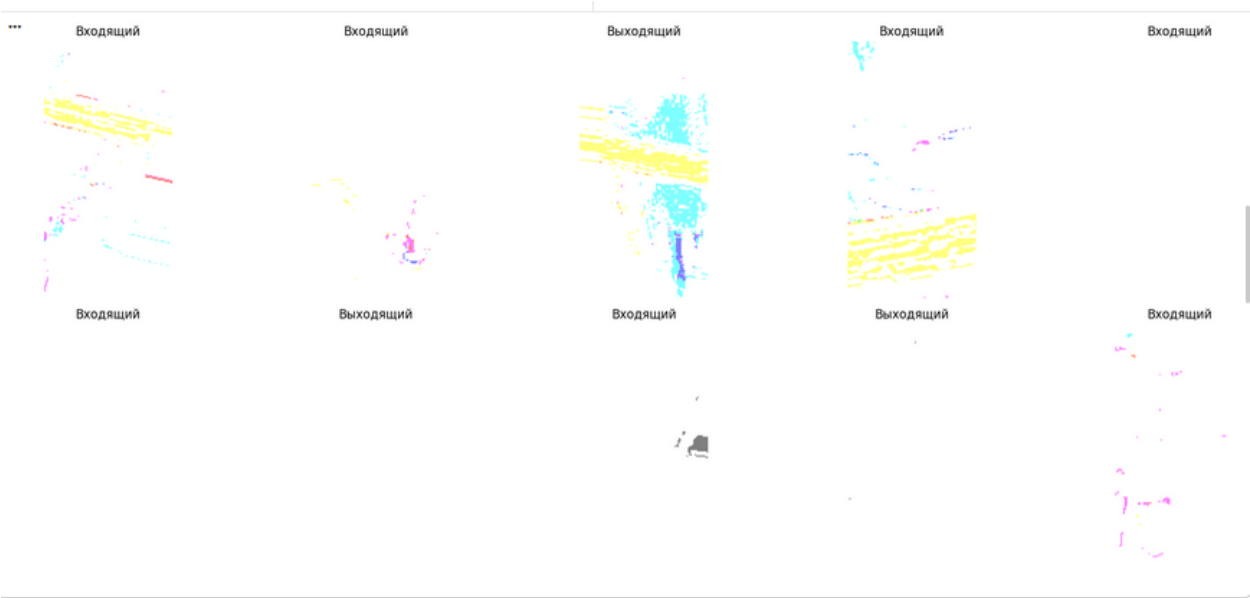


Рисунок 2.12 – Пример батча аугментированных изображений

2.12 Выбор базовой модели.

Базовая модель загружается с весами ImageNet, но без финального классификатора, после чего её веса полностью «замораживаются» для первого этапа обучения. Поверх базы через Functional API надстраивается новая «голова», включающая слои глобального усреднения, нормализации, полносвязный слой со 128 нейронами и Dropout для защиты от переобучения.

```
# Загружаем предобученную EfficientNetB0
# include_top=False — без её родного классификатора на 1000 классов ImageNet
base_model = EfficientNetB0(
    weights='imagenet',
    include_top=False,
    input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS),
)

#  ЭТАП 1: замораживаем всю базу
base_model.trainable = False

#  Собираем модель через Functional API
inputs = Input(shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS))

# training=False важно для замороженной базы:
# слои BatchNorm (а в EfficientNet их много) останутся в режиме inference
# и не будут обновлять свою статистику.
x = base_model(inputs, training=False)

# Голова модели
x = GlobalAveragePooling2D()(x)
x = BatchNormalization()(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)
outputs = Dense(CLASS_COUNT, activation='softmax')(x)

cnn_model = Model(inputs, outputs)
cnn_model.summary()

# Подсчёт параметров
trainable     = sum([np.prod(v.shape) for v in cnn_model.trainable_weights])
non_trainable = sum([np.prod(v.shape) for v in cnn_model.non_trainable_weights])
print(f'\nОбучаемых параметров (этап 1):     {trainable:,}')
print(f'Замороженных параметров (этап 1):   {non_trainable:,}')
```

Рисунок 2.13 – Архитектура свёрточной нейронной сети

Общее количество параметров составляет более 4.2 миллионов, из которых на первом этапе обучаемыми остаются только 166 786 параметров финальных слоев. Это подтверждает стратегию Transfer Learning, где основная мощь предобученной сети используется как фиксированный экстрактор признаков.

```
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0_notop.h5
16705208/16705208 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Model: "functional"
```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 192, 96, 3)	0
efficientnetb0 (Functional)	(None, 6, 3, 1280)	4,049,571
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
batch_normalization (BatchNormalization)	(None, 1280)	5,120
dense (Dense)	(None, 128)	163,968
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 2)	258

```
Total params: 4,218,917 (16.09 MB)
Trainable params: 166,786 (651.51 KB)
Non-trainable params: 4,052,131 (15.46 MB)

Обучаемых параметров (этап 1):     166,786
Замороженных параметров (этап 1):   4,052,131.0
```

Рисунок 2.14 – Структура модели

2.13 Обучение модели.

Использованы колбэки EarlyStopping для предотвращения бесполезных циклов при стагнации точности. Также здесь начинается разморозка специфических блоков (block6 и block7) базовой модели для последующей тонкой настройки, при этом слои BatchNormalization остаются замороженными для сохранения стабильной статистики.

После стабилизации «головы» была проведена разморозка последних блоков базовой модели (block6 и block7). Обучение продолжено с очень низким коэффициентом скорости обучения ($lr = 1e-05$), чтобы аккуратно подстроить глубокие фильтры под специфику текущих изображений, не разрушая предобученные веса.

Листинг 2.1 – Обучение модели

```
# Колбэки
early_stop = EarlyStopping(
    monitor='val_accuracy',
    patience=7,
    restore_best_weights=True,
    verbose=1,
)

lr_reducer = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.2,
    patience=3,
    min_lr=1e-7,
    verbose=1,
)

#  ЭТАП 1: обучение головы (база заморожена)
cnn_model.compile(
    optimizer=Adam(learning_rate=LR_STAGE1),
    loss='categorical_crossentropy',
    metrics=['accuracy'],
)


print('ЭТАП 1: обучение головы (база EfficientNetB0 заморожена)')
history1 = cnn_model.fit(
    train_generator,
    epochs=EPOCHS_STAGE1,
    validation_data=validation_generator,
    class_weight=class_weight_dict,
    callbacks=[early_stop, lr_reducer],
    verbose=1,
)

#  ЭТАП 2: размораживаем block6 и block7
# Сначала размораживаем всю базу...
base_model.trainable = True

# ...а потом замораживаем обратно всё, КРОМЕ block6 и block7.
# Дополнительно: BatchNorm-слои оставляем в режиме inference (frozen) -
# это стандартная практика для fine-tuning EfficientNet, иначе статистика
BN
# может «съехать» на маленьком батче и испортить предсказания.
for layer in base_model.layers:
    if not (layer.name.startswith('block6') or
layer.name.startswith('block7')):
        layer.trainable = False
    if isinstance(layer, BatchNormalization):
        layer.trainable = False
```

```
# Перекомпилируем с очень маленьким lr
cnn_model.compile(
    optimizer=Adam(learning_rate=LR_STAGE2),
    loss='categorical_crossentropy',
    metrics=['accuracy'],
)

trainable = sum([np.prod(v.shape) for v in cnn_model.trainable_weights])
print(f'\nОбучаемых параметров (этап 2): {trainable:,}')

# Сбрасываем early_stop, чтобы patience считался заново для второго этапа
early_stop = EarlyStopping(
    monitor='val_accuracy', patience=7,
    restore_best_weights=True, verbose=1,
)
lr_reducer = ReduceLROnPlateau(
    monitor='val_loss', factor=0.2, patience=3,
    min_lr=1e-7, verbose=1,
)

print()
print(f'ЭТАП 2: fine-tuning block6+block7 (lr = {LR_STAGE2})')
history2 = cnn_model.fit(
    train_generator,
    epochs=EPOCHS_STAGE2,
    validation_data=validation_generator,
    class_weight=class_weight_dict,
    callbacks=[early_stop, lr_reducer],
    verbose=1,
)

# Склеиваем истории двух этапов в одну
history = type('H', (), {})()  # пустой объект, чтобы дальше использовать
history.history как раньше
history.history = {
    'accuracy':     history1.history['accuracy']     +
history2.history['accuracy'],
    'val_accuracy': history1.history['val_accuracy'] +
history2.history['val_accuracy'],
    'loss':         history1.history['loss']         +
history2.history['loss'],
    'val_loss':     history1.history['val_loss']     +
history2.history['val_loss'],
}

# Запоминаем границу для графика
STAGE_BOUNDARY = len(history1.history['accuracy']) - 0.5
```

```
205/205 ━━━━━━━━━━━━━━━━━━━━ 22s 105ms/step - accuracy: 0.8400 - loss: 0.3319 - val_accuracy: 0.8813 - val_loss: 0.2581 - learning_rate: 1.0000e-04
Epoch 14/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 99ms/step - accuracy: 0.8493 - loss: 0.3183 - val_accuracy: 0.9131 - val_loss: 0.2090 - learning_rate: 2.0000e-05
Epoch 15/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 22s 105ms/step - accuracy: 0.8533 - loss: 0.3093 - val_accuracy: 0.9119 - val_loss: 0.2112 - learning_rate: 2.0000e-05
Epoch 16/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 22s 107ms/step - accuracy: 0.8484 - loss: 0.3122 - val_accuracy: 0.9138 - val_loss: 0.2039 - learning_rate: 2.0000e-05
Epoch 17/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 100ms/step - accuracy: 0.8541 - loss: 0.3045 - val_accuracy: 0.9168 - val_loss: 0.2018 - learning_rate: 2.0000e-05
Epoch 18/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 103ms/step - accuracy: 0.8548 - loss: 0.3103 - val_accuracy: 0.9205 - val_loss: 0.1992 - learning_rate: 2.0000e-05
Epoch 19/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 22s 105ms/step - accuracy: 0.8606 - loss: 0.3003 - val_accuracy: 0.9187 - val_loss: 0.1921 - learning_rate: 2.0000e-05
Epoch 20/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 98ms/step - accuracy: 0.8573 - loss: 0.3121 - val_accuracy: 0.9187 - val_loss: 0.1974 - learning_rate: 2.0000e-05
Epoch 21/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 102ms/step - accuracy: 0.8657 - loss: 0.2909 - val_accuracy: 0.9174 - val_loss: 0.2009 - learning_rate: 2.0000e-05
Epoch 22/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 0s 98ms/step - accuracy: 0.8647 - loss: 0.2999
Epoch 22: ReduceLROnPlateau reducing learning rate to 1e-05.
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 101ms/step - accuracy: 0.8582 - loss: 0.2971 - val_accuracy: 0.9156 - val_loss: 0.1926 - learning_rate: 2.0000e-05
Epoch 23/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 96ms/step - accuracy: 0.8665 - loss: 0.2911 - val_accuracy: 0.9223 - val_loss: 0.1902 - learning_rate: 1.0000e-05
Epoch 24/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 102ms/step - accuracy: 0.8663 - loss: 0.2911 - val_accuracy: 0.9229 - val_loss: 0.1839 - learning_rate: 1.0000e-05
Epoch 25/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 99ms/step - accuracy: 0.8717 - loss: 0.2880 - val_accuracy: 0.9229 - val_loss: 0.1838 - learning_rate: 1.0000e-05
Epoch 26/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 103ms/step - accuracy: 0.8649 - loss: 0.2922 - val_accuracy: 0.9223 - val_loss: 0.1832 - learning_rate: 1.0000e-05
Epoch 27/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 104ms/step - accuracy: 0.8634 - loss: 0.2929 - val_accuracy: 0.9193 - val_loss: 0.1823 - learning_rate: 1.0000e-05
Epoch 28/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 98ms/step - accuracy: 0.8603 - loss: 0.2909 - val_accuracy: 0.9254 - val_loss: 0.1840 - learning_rate: 1.0000e-05
Epoch 29/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 101ms/step - accuracy: 0.8683 - loss: 0.2875 - val_accuracy: 0.9205 - val_loss: 0.1848 - learning_rate: 1.0000e-05
Epoch 30/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 97ms/step - accuracy: 0.8613 - loss: 0.2918 - val_accuracy: 0.9217 - val_loss: 0.1828 - learning_rate: 1.0000e-05
Epoch 31/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 99ms/step - accuracy: 0.8707 - loss: 0.2808 - val_accuracy: 0.9223 - val_loss: 0.1814 - learning_rate: 1.0000e-05
Epoch 32/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 101ms/step - accuracy: 0.8672 - loss: 0.2842 - val_accuracy: 0.9248 - val_loss: 0.1816 - learning_rate: 1.0000e-05
Epoch 33/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 97ms/step - accuracy: 0.8629 - loss: 0.2951 - val_accuracy: 0.9248 - val_loss: 0.1847 - learning_rate: 1.0000e-05
Epoch 34/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 102ms/step - accuracy: 0.8634 - loss: 0.2871 - val_accuracy: 0.9291 - val_loss: 0.1766 - learning_rate: 1.0000e-05
Epoch 35/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 100ms/step - accuracy: 0.8697 - loss: 0.2867 - val_accuracy: 0.9272 - val_loss: 0.1739 - learning_rate: 1.0000e-05
Epoch 36/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 99ms/step - accuracy: 0.8709 - loss: 0.2817 - val_accuracy: 0.9266 - val_loss: 0.1737 - learning_rate: 1.0000e-05
Epoch 37/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 102ms/step - accuracy: 0.8707 - loss: 0.2788 - val_accuracy: 0.9205 - val_loss: 0.1873 - learning_rate: 1.0000e-05
Epoch 38/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 97ms/step - accuracy: 0.8668 - loss: 0.2833 - val_accuracy: 0.9278 - val_loss: 0.1684 - learning_rate: 1.0000e-05
Epoch 39/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 104ms/step - accuracy: 0.8680 - loss: 0.2853 - val_accuracy: 0.9248 - val_loss: 0.1779 - learning_rate: 1.0000e-05
Epoch 40/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 21s 102ms/step - accuracy: 0.8657 - loss: 0.2849 - val_accuracy: 0.9229 - val_loss: 0.1839 - learning_rate: 1.0000e-05
Epoch 41/50
205/205 ━━━━━━━━━━━━━━━━━━━━ 20s 98ms/step - accuracy: 0.8723 - loss: 0.2786 - val_accuracy: 0.9254 - val_loss: 0.1756 - learning_rate: 1.0000e-05
Epoch 41: early stopping
Restoring model weights from the end of the best epoch: 34.
```

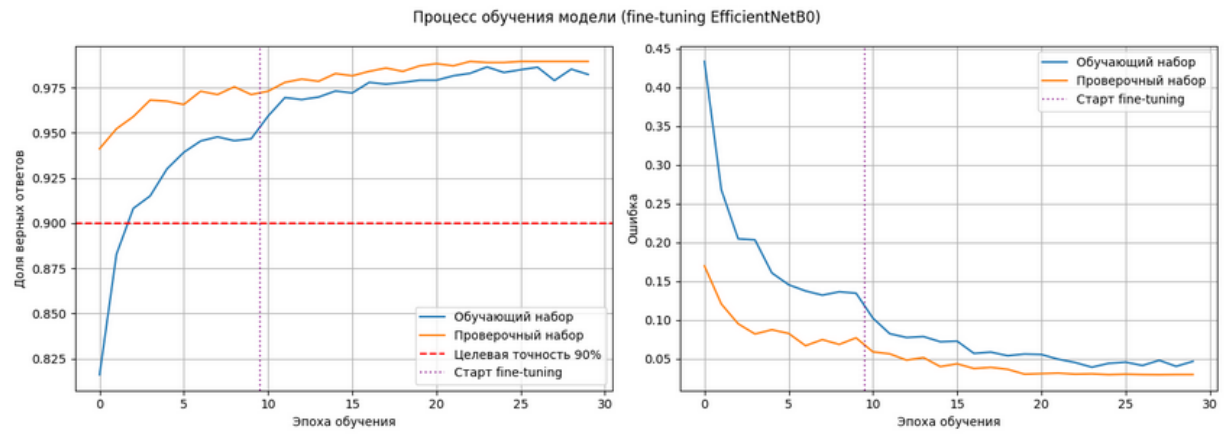
Рисунок 2.15 – Логи обучения по эпохам

2.14 Построение графика обучения.

Визуализация подтверждает, что целевой показатель точности в 90% был успешно достигнут и значительно превышен, составив 98.96% на проверочной выборке. Графики наглядно показывают момент перехода к Fine-tuning (пунктирная линия) и отсутствие признаков явного переобучения модели.

```python
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Процесс обучения модели (fine-tuning EfficientNetB0)')

ax1.plot(history.history['accuracy'],     label='Обучающий набор')
ax1.plot(history.history['val_accuracy'], label='Проверочный набор')
ax1.axhline(y=0.9, color='r', linestyle='--', label='Целевая точность 90%')
ax1.axvline(x=STAGE_BOUNDARY, color='purple', linestyle=':', alpha=0.7, label='Старт fine-tuning')
ax1.set_xlabel('Эпоха обучения')
ax1.set_ylabel('Доля верных ответов')
ax1.legend()
ax1.grid(True)

ax2.plot(history.history['loss'],     label='Обучающий набор')
ax2.plot(history.history['val_loss'], label='Проверочный набор')
ax2.axvline(x=STAGE_BOUNDARY, color='purple', linestyle=':', alpha=0.7, label='Старт fine-tuning')
ax2.set_xlabel('Эпоха обучения')
ax2.set_ylabel('Ошибка')
ax2.legend()
ax2.grid(True)

plt.tight_layout()
plt.show()

best_val_acc = max(history.history['val_accuracy'])
print(f'Лучшая точность на проверочной выборке: {best_val_acc*100:.2f}%')
print(f'Целевая точность 90% {"ДОСТИГНУТА" if best_val_acc >= 0.9 else "НЕ достигнута"}')
```

Рисунок 2.16 – Код построения графика обучения




```
Лучшая точность на проверочной выборке: 98.96%
Целевая точность 90% ДОСТИГНУТА
```

Рисунок 2.17 – Графики процесса обучения модели

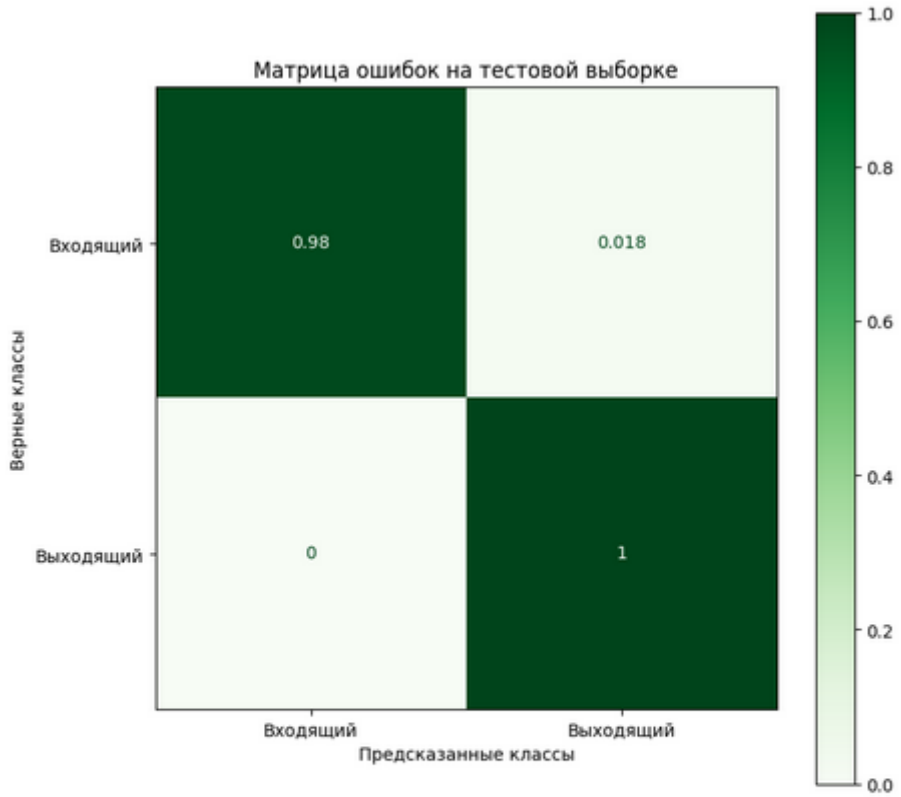
2.15 Оценка качества модели на тестовой выборке.

Матрица ошибок демонстрирует почти идеальное распределение: модель безошибочно классифицирует «Выходящих» (100%) и лишь в редких случаях (1.8%) ошибается на «Входящих». Итоговые метрики F1-score на уровне 0.98–0.99 подтверждают высокую обобщающую способность алгоритма.

```python
# Получаем предсказания на ВСЕЙ тестовой выборке (а не на одном батче)
test_generator.reset()
y_pred = cnn_model.predict(test_generator)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = y_test  # исходные метки до OHE

# Матрица ошибок (нормализованная по строкам)
cm = confusion_matrix(y_true, y_pred_classes, normalize='true')
cm = np.round(cm, 3)

fig, ax = plt.subplots(figsize=(8, 8))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASS_LIST)
disp.plot(ax=ax, cmap='Greens', colorbar=True)
plt.title('Матрица ошибок на тестовой выборке')
plt.xlabel('Предсказанные классы')
plt.ylabel('Верные классы')
plt.show()

# Подробный отчёт: precision, recall, F1 по каждому классу
print('\nПодробный отчёт по классам:')
print(classification_report(y_true, y_pred_classes, target_names=CLASS_LIST))
```

Рисунок 2.18 – Код оценки модели на тестовой выборке

```
29/29 ━━━━━━━━━━━━━━━━━━━━ 21s 491ms/step
```




```
Подробный отчёт по классам:
              precision    recall  f1-score   support

    Входящий       1.00      0.98      0.99       649
   Выходящий       0.96      1.00      0.98       260

    accuracy                           0.99       909
   macro avg       0.98      0.99      0.98       909
weighted avg       0.99      0.99      0.99       909
```

Рисунок 2.19 – Матрица ошибок и подробный отчёт по классам

2.16 Визуальный просмотр предсказаний.

Для качественной оценки работы модели выводится сетка из 10 случайных изображений тестовой выборки с указанием истинного и предсказанного класса, а также уверенности модели. Заголовки изображений окрашены в зелёный цвет при правильном предсказании и в красный – при ошибке, что позволяет наглядно увидеть характер допускаемых ошибок.

```
# Визуальный просмотр предсказаний на случайных примерах.
# x_test ещё в исходном виде (0..255), preprocess_input применяется в test_generator.
fig, axs = plt.subplots(2, 5, figsize=(20, 8))
axs = axs.flatten()

indices = random.sample(range(len(x_test)), 10)
for i, idx in enumerate(indices):
    img = (x_test[idx] / 255.0).clip(0, 1)
    true_label = CLASS_LIST[y_test[idx]]
    pred_label = CLASS_LIST[y_pred_classes[idx]]
    pred_proba = y_pred[idx][y_pred_classes[idx]]

    color = 'green' if true_label == pred_label else 'red'
    axs[i].imshow(img)
    axs[i].set_title(f'Истина: {true_label}\nПредсказано: {pred_label} ({pred_proba:.2f})',
                     color=color, fontsize=10)
    axs[i].axis('off')

plt.tight_layout()
plt.show()
```

Рисунок 2.20 – Код визуального просмотра предсказаний

Рисунок 2.21 – Примеры предсказаний модели на случайных изображениях

Выводы: Применение двухэтапного процесса обучения – от тренировки только выходного классификатора до тонкой настройки (Fine-tuning) глубоких слоев – обеспечило высокую стабильность и точность предсказаний. Внедрение механизмов аугментации данных и взвешивания классов помогло успешно преодолеть естественный дисбаланс выборки, сделав модель устойчивой к различным условиям съемки и освещения.

Итоговые результаты демонстрируют достижение целевых показателей: точность на проверочной выборке составила 98.96%, что подтверждается графиками обучения и матрицей ошибок на тестовых данных. Высокие значения метрик Precision и Recall (в диапазоне 0.98–1.00) и успешное визуальное тестирование подтверждают готовность системы к практической эксплуатации с минимальным уровнем ложных срабатываний.

Задание 3. Решить задачу классификации автомобилей по маркам с применением технологии Transfer Learning.

Цель – добиться точности распознавания не ниже 93% на проверочной выборке. В качестве предобученной базы используется модель MobileNetV2, обученная на датасете ImageNet (~1.4 млн изображений). База данных задачи – middle_fmr.zip из ресурсов УИИ, содержащая изображения трёх марок: Ferrari, Mercedes, Renault.

Идея Transfer Learning состоит в том, чтобы взять модель, уже обученную решать общую задачу распознавания изображений, и дообучить её последние слои под конкретную задачу. Это позволяет получить высокую точность на относительно небольших датасетах за короткое время обучения, поскольку низкоуровневые признаки (границы, текстуры, простые формы) уже выучены базой и не требуют переобучения.

3.1 Подключение библиотек.

Помимо стандартных библиотек numpy, matplotlib, PIL, gdown и инструментов scikit-learn (train_test_split, compute_class_weight, метрики), импортируются модули TensorFlow / Keras для построения функциональной модели: Model из keras.models, слои Dense, Dropout, GlobalAveragePooling2D, BatchNormalization, Input, генератор аугментации ImageDataGenerator и колбэки EarlyStopping и ReduceLROnPlateau. Ключевыми для Transfer Learning являются импорты MobileNetV2 из tensorflow.keras.applications и функции preprocess_input из соответствующего модуля mobilenet_v2 – она выполняет

нормализацию входных данных в формат, ожидаемый предобученной моделью.

Классификация автомобилей с использованием Transfer Learning

Цель: добиться точности распознавания не ниже 93% на проверочной выборке.

Подход: transfer learning на базе предобученной модели MobileNetV2 из `tf.keras.applications`.

База данных: middle_fmr.zip (база автомобилей УИИ).

Идея transfer learning: взять модель, уже обученную на ImageNet (~1.4 млн изображений), и дообучить её последние слои под нашу задачу. Это даёт высокую точность на маленьких датасетах за короткое время обучения.

Загрузка библиотек

[1]:
```
# Базовые библиотеки
import os
import random
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import gdown

# sklearn
from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report

# Keras / TensorFlow
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import (Dense, Dropout, GlobalAveragePooling2D,
                                     BatchNormalization, Input)
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam

# Предобученная архитектура
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
```

Рисунок 3.1 – Подключение библиотек и предобученной архитектуры

3.2 Загрузка датасета.

Архив с изображениями автомобилей (middle_fmr.zip) загружается с облачного хранилища УИИ при помощи gdown. После загрузки выполняется проверка размера файла: если архив подозрительно мал (меньше 1 МБ), выбрасывается AssertionError, что предохраняет от продолжения работы с повреждённым архивом. Затем архив распаковывается в директорию /content/cars и выводится список папок-классов: Ferrari, Mercedes, Renault.

Загрузка датасета

In [2]:

```
# Загрузка zip-архива из облака УИИ
DATASET_URL = 'https://storage.yandexcloud.net/aiueducation/Content/base/l5/middle_fmr.zip'
DATASET_ZIP = 'middle_fmr.zip'

gdown.download(DATASET_URL, DATASET_ZIP, quiet=False)

# Проверка, что файл скачался корректно
size_mb = os.path.getsize(DATASET_ZIP) / 1024 / 1024
print(f'\nРазмер скачанного архива: {size_mb:.1f} МБ')
assert size_mb > 1, 'Архив подозрительно маленький, проверьте URL!'

# Распаковка архива
!unzip -q -o "{DATASET_ZIP}" -d /content/cars

# Папка с папками-классами
IMAGE_PATH = '/content/cars'
print('\nСодержимое:', os.listdir(IMAGE_PATH))
```

```
Downloading...
From: https://storage.yandexcloud.net/aiueducation/Content/base/l5/middle_fmr.zip
To: /content/middle_fmr.zip
100%|██████████| 119M/119M [00:05<00:00, 20.4MB/s]
Размер скачанного архива: 113.5 МБ

Содержимое: ['Ferrari', 'Mercedes', 'Renault']
```

Рисунок 3.2 – Загрузка и распаковка датасета middle_fmr.zip

3.3 Определение списка классов.

При наличии вложенной структуры архива (если внутри корневой директории оказывается единственная папка) производится автоматическая корректировка пути к классам. Список классов формируется только из директорий (для устойчивости к посторонним файлам в корне), сортируется по алфавиту и подсчитывается размер каждого класса. Датасет содержит три класса: Ferrari (1088 фото), Mercedes (1161 фото), Renault (1178 фото). Распределение почти сбалансировано – соотношение классов близко к 1:1:1.

```
In [3]:
# Если внутри один уровень вложенности - корректируем путь
contents = os.listdir(IMAGE_PATH)
if len(contents) == 1 and os.path.isdir(os.path.join(IMAGE_PATH, contents[0])):
    IMAGE_PATH = os.path.join(IMAGE_PATH, contents[0])
    print(f'Путь скорректирован: {IMAGE_PATH}')

# Список классов
CLASS_LIST = sorted([d for d in os.listdir(IMAGE_PATH)
                     if os.path.isdir(os.path.join(IMAGE_PATH, d))])
CLASS_COUNT = len(CLASS_LIST)

print(f'\nКоличество классов: {CLASS_COUNT}')
print(f'Метки классов: {CLASS_LIST}')

# Размер каждого класса
print('\nРазмер каждого класса:')
for cls in CLASS_LIST:
    n = len(os.listdir(os.path.join(IMAGE_PATH, cls)))
    print(f'  {cls}: {n} фото')

Количество классов: 3
Метки классов: ['Ferrari', 'Mercedes', 'Renault']

Размер каждого класса:
  Ferrari: 1088 фото
  Mercedes: 1161 фото
  Renault: 1178 фото
```

Рисунок 3.3 – Список классов и количество изображений в каждом

3.4 Гиперпараметры.

Параметры разделения выборки те же, что и в Задании 2: 10% – тестовая выборка, 20% от обучающего набора – проверочная. Размер изображения выбран 224×224 пикселя – это стандартный входной размер для предобученной модели MobileNetV2, при котором она показывает максимальную точность на ImageNet.

Обучение разбито на два этапа: на первом этапе (10 эпох) обучается только новая «голова» модели – верхние полносвязные слои – при замороженной базе MobileNetV2; на втором этапе (20 эпох) выполняется fine-tuning – размораживается часть верхних слоёв базы и вся модель дообучается с очень малым learning rate. Размер батча – 32.

Гиперпараметры

In [4]:
```
# Параметры разделения
TEST_SPLIT      = 0.1
VAL_SPLIT       = 0.2

# Размер изображения - 224x224 стандарт для MobileNetV2
IMG_WIDTH       = 224
IMG_HEIGHT      = 224
IMG_CHANNELS    = 3

# Параметры обучения
BATCH_SIZE      = 32
EPOCHS_FROZEN   = 10    # Этап 1: обучаем только голову
EPOCHS_FINETUNE = 20    # Этап 2: fine-tuning
```

Рисунок 3.4 – Заданные гиперпараметры

3.5 Загрузка изображений.

Изображения последовательно загружаются из всех директорий классов с приведением к размеру 224×224. По сравнению с Заданием 2 добавлен блок try/except: если файл не удалось загрузить (например, из-за повреждения или нестандартного формата), выводится сообщение и файл пропускается, не прерывая работу всего скрипта. Всего успешно загружено 3427 изображений, форма массива (3427, 224, 224, 3), распределение классов: 1088 / 1161 / 1178.

Загрузка изображений

In [5]:
```
images = []
labels = []

for i, class_name in enumerate(CLASS_LIST):
    class_path = os.path.join(IMAGE_PATH, class_name)
    for img_name in os.listdir(class_path):
        img_path = os.path.join(class_path, img_name)
        try:
            img = image.load_img(img_path, target_size=(IMG_HEIGHT, IMG_WIDTH))
            images.append(image.img_to_array(img))
            labels.append(i)
        except Exception as e:
            print(f'Пропущен {img_path}: {e}')

images = np.array(images, dtype='float32')
labels = np.array(labels)

print(f'Всего изображений: {len(images)}')
print(f'Форма массива: {images.shape}')
print(f'Распределение классов: {np.bincount(labels)}')
```

```
Всего изображений: 3427
Форма массива: (3427, 224, 224, 3)
Распределение классов: [1088 1161 1178]
```

Рисунок 3.5 – Загрузка изображений и статистика по выборке

3.6 Просмотр примеров изображений.

Для ознакомления с содержимым датасета выводится по одному изображению из каждого класса. Видно, что в датасете присутствуют качественные студийные и рекламные снимки автомобилей в разных условиях освещения и ракурсах. Это существенно отличает датасет от Задания 2, где использовались кадры с камер видеонаблюдения.

```
In [6]: # Просмотр по одному изображению из каждого класса
        fig, axs = plt.subplots(1, CLASS_COUNT, figsize=(4*CLASS_COUNT, 4))
        if CLASS_COUNT == 1:
            axs = [axs]

        for i in range(CLASS_COUNT):
            idx = np.where(labels == i)[0][0]
            axs[i].imshow(images[idx].astype('uint8'))
            axs[i].set_title(CLASS_LIST[i])
            axs[i].axis('off')
        plt.tight_layout()
        plt.show()
```

Рисунок 3.6 – Примеры изображений каждого класса

3.7 Разделение на train / val / test со стратификацией.

Разделение производится по той же двухэтапной схеме со стратификацией, что и в Задании 2. В результате получены три выборки: обучающая – 2467 изображений, проверочная – 617, тестовая – 343. Все массивы имеют форму (n, 224, 224, 3), метки переведены в формат One-Hot Encoding.

Разделение на train / val / test со стратификацией

```
In [7]:
# Сначала отделяем тестовую выборку
x_train_full, x_test, y_train_full, y_test = train_test_split(
    images, labels,
    test_size=TEST_SPLIT,
    random_state=42,
    stratify=labels
)

# Затем делим на обучающую и проверочную
x_train, x_val, y_train_raw, y_val_raw = train_test_split(
    x_train_full, y_train_full,
    test_size=VAL_SPLIT,
    random_state=42,
    stratify=y_train_full
)

# OHE
y_train = to_categorical(y_train_raw, num_classes=CLASS_COUNT)
y_val = to_categorical(y_val_raw, num_classes=CLASS_COUNT)
y_test_ohe = to_categorical(y_test, num_classes=CLASS_COUNT)

print(f'Обучающий набор:    {x_train.shape}')
print(f'Проверочный набор:  {x_val.shape}')
print(f'Тестовый набор:     {x_test.shape}')

Обучающий набор:    (2467, 224, 224, 3)
Проверочный набор:  (617, 224, 224, 3)
Тестовый набор:     (343, 224, 224, 3)
```

Рисунок 3.7 – Разделение выборки на train, val и test

3.8 Расчёт весов классов.

Хотя распределение классов в исходных данных близко к равномерному (32% / 34% / 34%), для устойчивости работы compute_class_weight в режиме «balanced» применяется и здесь. Полученные веса: 1.060 для Ferrari, 0.984 для Mercedes, 0.970 для Renault – веса близки к 1, что подтверждает практическую сбалансированность датасета. Тем не менее их передача в model.fit гарантирует корректное взвешивание ошибок при минимальных колебаниях частот в батчах.

```
In [8]:
# Веса классов на случай дисбаланса
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_train_raw),
    y=y_train_raw
)
class_weight_dict = dict(enumerate(class_weights))
print(f'Веса классов: {class_weight_dict}')

Веса классов: {0: np.float64(1.0502341421881651), 1: np.float64(0.983652312599681), 2: np.float64(0.9697327044025157)}
```

Рисунок 3.8 – Расчёт весов классов

3.9 Аугментация и подготовка данных.

Важная особенность работы с предобученными моделями: вместо стандартной нормализации rescale=1./255 необходимо использовать функцию preprocess_input из tensorflow.keras.applications.mobilenet_v2. Эта функция

приводит данные к диапазону [-1, 1], в котором обучалась MobileNetV2 на ImageNet – без такого согласования предобученные веса работают некорректно.

Параметры аугментации скорректированы под специфику задачи: повороты ±15°, сдвиги ±10%, диапазон масштабирования увеличен до ±15%, диапазон яркости расширен до (0.8, 1.2), включено горизонтальное отражение (horizontal_flip=True) – в отличие от задачи о пассажирах, для классификации марки автомобиля зеркальное отражение не меняет класса. Параметр fill_mode='nearest' определяет стратегию заполнения пикселей при сдвигах и поворотах.

Аугментация и подготовка данных

Важно: для предобученных моделей вместо стандартной нормализации `1/255` используется `preprocess_input` из `tf.keras.applications.mobilenet_v2`. Эта функция приводит данные к диапазону [-1, 1], в котором обучалась MobileNetV2.

In [9]:
```
# Генератор для обучения - аугментация + нормализация под MobileNetV2
train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.15,
    brightness_range=(0.8, 1.2),
    horizontal_flip=True,
    fill_mode='nearest'
)

# Для val и test - только нормализация
val_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)
test_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)

train_generator = train_datagen.flow(x_train, y_train, batch_size=BATCH_SIZE, shuffle=True)
validation_generator = val_datagen.flow(x_val, y_val, batch_size=BATCH_SIZE, shuffle=False)
test_generator = test_datagen.flow(x_test, y_test_ohe, batch_size=BATCH_SIZE, shuffle=False)
```

Рисунок 3.9 – Создание генераторов с preprocess_input и аугментацией

3.10 Создание модели на базе MobileNetV2.

Архитектура итоговой модели состоит из четырёх частей. Первая часть – предобученная база MobileNetV2 с параметром include_top=False (без верхнего classification-слоя ImageNet) и weights='imagenet' (загружаются предобученные веса). На этапе создания модели база замораживается (base_model.trainable = False) – её 2 260 800 параметров не будут обучаться на первом этапе.

Вторая часть – GlobalAveragePooling2D, преобразующий карту признаков размера (7, 7, 1280) в вектор длиной 1280. Третья часть – новая «голова»: BatchNormalization, Dropout(0.3), Dense(128, relu), BatchNormalization, Dropout(0.4). Четвёртая часть – выходной Dense с активацией softmax по числу классов (3). Модель собирается через функциональный API Keras, что необходимо для последующей разморозки части базовых слоёв.

Создание модели на базе MobileNetV2

Архитектура:

1. **MobileNetV2** — предобученная база, без classification-головы
2. **GlobalAveragePooling2D** — преобразует карту признаков в вектор
3. **Dense + Dropout** — новая голова под нашу задачу
4. **Выходной Dense с softmax** — по числу классов

In [10]:

```
# Загружаем MobileNetV2 без верхнего слоя
base_model = MobileNetV2(
    input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS),
    include_top=False,
    weights='imagenet',
    pooling=None
)

# ЭТАП 1: замораживаем базу
base_model.trainable = False

# Собираем модель
inputs = Input(shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS))
x = base_model(inputs, training=False)
x = GlobalAveragePooling2D()(x)
x = BatchNormalization()(x)
x = Dropout(0.3)(x)
x = Dense(128, activation='relu')(x)
x = BatchNormalization()(x)
x = Dropout(0.4)(x)
outputs = Dense(CLASS_COUNT, activation='softmax')(x)

model = Model(inputs, outputs)

print(f'Всего параметров:       {model.count_params():,}')
print(f'Обучаемых параметров:   {sum(tf.size(w).numpy() for w in model.trainable_weights):,}')
print(f'Замороженных параметров: {sum(tf.size(w).numpy() for w in model.non_trainable_weights):,}')
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Всего параметров:       2,427,971
Обучаемых параметров:   167,171
Замороженных параметров: 2,260,800
```

Рисунок 3.10 – Создание модели на базе MobileNetV2

Общее количество параметров модели – 2 427 971, из которых обучаемых на первом этапе – всего 167 171 (только новая голова), замороженных – 2 260 800 (база MobileNetV2). Соотношение обучаемых к общему числу составляет около 7%, что является типичным для Transfer Learning и обеспечивает быстрое обучение головы.

3.11 Этап 1: обучение головы (база заморожена).

При замороженной базе можно использовать относительно высокий learning rate – обучаются только новые слои с случайной инициализацией. Параметры компиляции: оптимизатор Adam с шагом 0.001, функция ошибки categorical_crossentropy, метрика accuracy.

Для управления обучением настроены колбэки: EarlyStopping с patience=4 по val_accuracy и восстановлением лучших весов; ReduceLROnPlateau с factor=0.3, patience=2 и минимальным шагом 1e-5 по val_loss. Этап 1 рассчитан на 10 эпох.

Этап 1: обучение головы (база заморожена)

С замороженной базой можем использовать высокий learning rate — обучаются только новые слои.

In [11]:
```
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

callbacks_stage1 = [
    EarlyStopping(monitor='val_accuracy', patience=4, restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=2, min_lr=1e-5, verbose=1)
]

history_stage1 = model.fit(
    train_generator,
    epochs=EPOCHS_FROZEN,
    validation_data=validation_generator,
    class_weight=class_weight_dict,
    callbacks=callbacks_stage1,
    verbose=1
)
```

```
Epoch 1/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 91s 890ms/step - accuracy: 0.6627 - loss: 0.9128 - val_accuracy: 0.8023 - val_loss: 0.4684 - learning_rate: 0.0010
Epoch 2/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 36s 459ms/step - accuracy: 0.7685 - loss: 0.6158 - val_accuracy: 0.8006 - val_loss: 0.4390 - learning_rate: 0.0010
Epoch 3/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 34s 435ms/step - accuracy: 0.7831 - loss: 0.5517 - val_accuracy: 0.8136 - val_loss: 0.4393 - learning_rate: 0.0010
Epoch 4/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 406ms/step - accuracy: 0.8058 - loss: 0.4939 - val_accuracy: 0.8233 - val_loss: 0.4135 - learning_rate: 0.0010
Epoch 5/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 424ms/step - accuracy: 0.8018 - loss: 0.4843 - val_accuracy: 0.8088 - val_loss: 0.4484 - learning_rate: 0.0010
Epoch 6/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 0s 389ms/step - accuracy: 0.8077 - loss: 0.4736
Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0003000000142492354.
78/78 ━━━━━━━━━━━━━━━━━━━━ 31s 400ms/step - accuracy: 0.8156 - loss: 0.4460 - val_accuracy: 0.8282 - val_loss: 0.4272 - learning_rate: 0.0010
Epoch 7/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 419ms/step - accuracy: 0.8285 - loss: 0.4079 - val_accuracy: 0.8282 - val_loss: 0.4213 - learning_rate: 3.0000e-04
Epoch 8/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 0s 387ms/step - accuracy: 0.8335 - loss: 0.4187
Epoch 8: ReduceLROnPlateau reducing learning rate to 9.000000427477062e-05.
78/78 ━━━━━━━━━━━━━━━━━━━━ 31s 397ms/step - accuracy: 0.8322 - loss: 0.4129 - val_accuracy: 0.8314 - val_loss: 0.4161 - learning_rate: 3.0000e-04
Epoch 9/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 413ms/step - accuracy: 0.8253 - loss: 0.4176 - val_accuracy: 0.8331 - val_loss: 0.4087 - learning_rate: 9.0000e-05
Epoch 10/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 31s 394ms/step - accuracy: 0.8379 - loss: 0.3887 - val_accuracy: 0.8314 - val_loss: 0.4061 - learning_rate: 9.0000e-05
Restoring model weights from the end of the best epoch: 9.
```

Рисунок 3.11 – Код компиляции, обучение головы и логи этапа 1

Уже после первой эпохи val_accuracy достигает 0.8023 – это демонстрирует мощь Transfer Learning: благодаря качественным предобученным признакам новая голова быстро находит разделяющую гиперплоскость в пространстве признаков. Колбэк ReduceLROnPlateau сработал на 6-й и 8-й эпохах, последовательно понижая learning rate до 3e-4 и 9e-5. Этап 1 завершился восстановлением лучших весов 9-й эпохи (val_accuracy ≈ 0.83).

3.12 Этап 2: Fine-tuning (разморозка верхних слоёв базы).

Идея fine-tuning состоит в том, чтобы дообучить высокоуровневые слои предобученной базы под специфику нашей задачи, не разрушая полезные низкоуровневые признаки. Для этого размораживается вся база (base_model.trainable = True), но нижние слои затем явно замораживаются обратно: оставляются обучаемыми только верхние ~30% слоёв (с индекса 100 и выше).

Критически важно перекомпилировать модель после изменения trainable-флагов слоёв и использовать очень малый learning rate (1e-5) – это позволяет деликатно адаптировать признаки, не разрушая их шумными большими шагами градиента. После перекомпиляции число обучаемых параметров возрастает до 2 028 611 – почти на порядок больше, чем на этапе 1.

Этап 2: Fine-tuning (размораживаем верхние слои базы)

Размораживаем верхнюю часть MobileNetV2 и дообучаем всю модель с очень маленьким learning rate. Это позволяет модели адаптировать высокоуровневые признаки под автомобили, не разрушая полезные низкоуровневые признаки.

In [12]:

```
# Размораживаем базу
base_model.trainable = True

# Замораживаем нижние слои - учим только верхние ~30%
fine_tune_at = 100
for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

# Перекомпиляция с очень маленьким learning rate
model.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

trainable_params = sum(tf.size(w).numpy() for w in model.trainable_weights)
print(f'Обучаемых параметров на этапе 2: {trainable_params:,}')
```

```
Обучаемых параметров на этапе 2: 2,028,611
```

Рисунок 3.12 – Разморозка верхних слоёв и перекомпиляция модели

In [13]:

```
callbacks_stage2 = [
    EarlyStopping(monitor='val_accuracy', patience=6, restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=3, min_lr=1e-7, verbose=1)
]

history_stage2 = model.fit(
    train_generator,
    epochs=EPOCHS_FINETUNE,
    validation_data=validation_generator,
    class_weight=class_weight_dict,
    callbacks=callbacks_stage2,
    verbose=1
)
```

```
Epoch 1/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 78s 689ms/step - accuracy: 0.7471 - loss: 0.6327 - val_accuracy: 0.8282 - val_loss: 0.3980 - learning_rate: 1.0000e-05
Epoch 2/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 405ms/step - accuracy: 0.7783 - loss: 0.5441 - val_accuracy: 0.8104 - val_loss: 0.4581 - learning_rate: 1.0000e-05
Epoch 3/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 423ms/step - accuracy: 0.8139 - loss: 0.4750 - val_accuracy: 0.8055 - val_loss: 0.4909 - learning_rate: 1.0000e-05
Epoch 4/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 0s 514ms/step - accuracy: 0.8086 - loss: 0.4722
Epoch 4: ReduceLROnPlateau reducing learning rate to 2.9999999242136253e-06.
78/78 ━━━━━━━━━━━━━━━━━━━━ 41s 531ms/step - accuracy: 0.8079 - loss: 0.4767 - val_accuracy: 0.8169 - val_loss: 0.4760 - learning_rate: 1.0000e-05
Epoch 5/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 34s 427ms/step - accuracy: 0.8204 - loss: 0.4483 - val_accuracy: 0.8201 - val_loss: 0.4580 - learning_rate: 3.0000e-06
Epoch 6/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 404ms/step - accuracy: 0.8216 - loss: 0.4646 - val_accuracy: 0.8201 - val_loss: 0.4315 - learning_rate: 3.0000e-06
Epoch 7/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 0s 413ms/step - accuracy: 0.8064 - loss: 0.4753
Epoch 7: ReduceLROnPlateau reducing learning rate to 8.999999636216671e-07.
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 423ms/step - accuracy: 0.8200 - loss: 0.4572 - val_accuracy: 0.8298 - val_loss: 0.4146 - learning_rate: 3.0000e-06
Epoch 8/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 404ms/step - accuracy: 0.8273 - loss: 0.4266 - val_accuracy: 0.8363 - val_loss: 0.4050 - learning_rate: 9.0000e-07
Epoch 9/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 423ms/step - accuracy: 0.8212 - loss: 0.4369 - val_accuracy: 0.8444 - val_loss: 0.3913 - learning_rate: 9.0000e-07
Epoch 10/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 410ms/step - accuracy: 0.8314 - loss: 0.4091 - val_accuracy: 0.8509 - val_loss: 0.3823 - learning_rate: 9.0000e-07
Epoch 11/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 421ms/step - accuracy: 0.8306 - loss: 0.4418 - val_accuracy: 0.8525 - val_loss: 0.3743 - learning_rate: 9.0000e-07
Epoch 12/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 417ms/step - accuracy: 0.8245 - loss: 0.4560 - val_accuracy: 0.8525 - val_loss: 0.3669 - learning_rate: 9.0000e-07
Epoch 13/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 411ms/step - accuracy: 0.8379 - loss: 0.4055 - val_accuracy: 0.8590 - val_loss: 0.3630 - learning_rate: 9.0000e-07
Epoch 14/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 428ms/step - accuracy: 0.8229 - loss: 0.4372 - val_accuracy: 0.8541 - val_loss: 0.3573 - learning_rate: 9.0000e-07
Epoch 15/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 426ms/step - accuracy: 0.8456 - loss: 0.3968 - val_accuracy: 0.8525 - val_loss: 0.3537 - learning_rate: 9.0000e-07
Epoch 16/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 411ms/step - accuracy: 0.8419 - loss: 0.4126 - val_accuracy: 0.8525 - val_loss: 0.3509 - learning_rate: 9.0000e-07
Epoch 17/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 428ms/step - accuracy: 0.8362 - loss: 0.4043 - val_accuracy: 0.8509 - val_loss: 0.3494 - learning_rate: 9.0000e-07
Epoch 18/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 32s 409ms/step - accuracy: 0.8443 - loss: 0.3997 - val_accuracy: 0.8476 - val_loss: 0.3493 - learning_rate: 9.0000e-07
Epoch 19/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 33s 420ms/step - accuracy: 0.8293 - loss: 0.4189 - val_accuracy: 0.8444 - val_loss: 0.3488 - learning_rate: 9.0000e-07
Epoch 19: early stopping
Restoring model weights from the end of the best epoch: 13.
```

Рисунок 3.13 – Логи обучения на этапе fine-tuning

На этапе 2 настроены более терпеливые колбэки: EarlyStopping с patience=6 и ReduceLROnPlateau с patience=3 и min_lr=1e-7. Из логов видно, что точность на проверочной выборке плавно растёт от 0.8282 (1-я эпоха) до 0.859 (13-я эпоха). EarlyStopping сработал на 19-й эпохе и восстановил веса лучшей 13-й. ReduceLROnPlateau последовательно снижал learning rate (4-я эпоха – до 3e-6, 7-я эпоха – до 9e-7), что обеспечило плавную сходимость без срывов.

3.13 Построение графика обучения.

Поскольку обучение разбито на два этапа, история сохранена в двух разных объектах history_stage1 и history_stage2. Для построения единого графика их метрики последовательно объединяются (списки accuracy, val_accuracy, loss, val_loss из обоих этапов). На графиках дополнительно отображаются: горизонтальная пунктирная линия целевой точности 93% и вертикальная линия в момент перехода с этапа 1 на этап 2 (на границе stage1_end - 0.5), что позволяет визуально оценить эффект fine-tuning.

График обучения

In [14]:

```
# Объединяем историю обоих этапов
acc = history_stage1.history['accuracy'] + history_stage2.history['accuracy']
val_acc = history_stage1.history['val_accuracy'] + history_stage2.history['val_accuracy']
loss = history_stage1.history['loss'] + history_stage2.history['loss']
val_loss = history_stage1.history['val_loss'] + history_stage2.history['val_loss']

stage1_end = len(history_stage1.history['accuracy'])

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Процесс обучения')

ax1.plot(acc, label='Обучающий набор')
ax1.plot(val_acc, label='Проверочный набор')
ax1.axhline(y=0.93, color='r', linestyle='--', alpha=0.7, label='Целевая точность 93%')
ax1.axvline(x=stage1_end - 0.5, color='gray', linestyle=':', alpha=0.7, label='Начало fine-tuning')
ax1.set_xlabel('Эпоха')
ax1.set_ylabel('Доля верных ответов')
ax1.legend()
ax1.grid(True)

ax2.plot(loss, label='Обучающий набор')
ax2.plot(val_loss, label='Проверочный набор')
ax2.axvline(x=stage1_end - 0.5, color='gray', linestyle=':', alpha=0.7, label='Начало fine-tuning')
ax2.set_xlabel('Эпоха')
ax2.set_ylabel('Ошибка')
ax2.legend()
ax2.grid(True)

plt.tight_layout()
plt.show()

best_val_acc = max(val_acc)
print(f'\nЛучшая точность на проверочной выборке: {best_val_acc:.4f} ({best_val_acc*100:.2f}%)')
if best_val_acc >= 0.93:
    print('Цель достигнута! Точность не ниже 93%')
else:
    print('Цель пока не достигнута')
```

Рисунок 3.14 – Код построения объединённого графика обучения

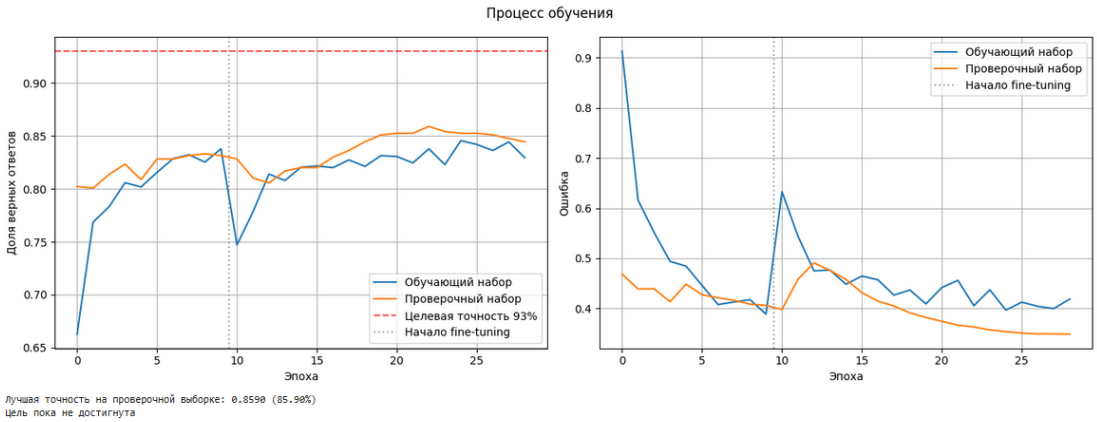


Рисунок 3.15 – Графики процесса обучения по обоим этапам

На графике accuracy чётко видны два этапа обучения, разделённые серой пунктирной линией. На первом этапе (0–9 эпохи, база заморожена) val_accuracy быстро поднимается с 0.80 до плато около 0.83 – это эффект Transfer Learning, когда новая голова за несколько эпох настраивается под уже выученные базой признаки. В момент перехода на второй этап (fine-tuning) видна резкая просадка train_accuracy до 0.74 и скачок train_loss – это типичное поведение при разморозке слоёв базы и пересчёте оптимизатора с нуля на очень малом learning rate. После 12-й эпохи модель плавно восстанавливается и продолжает медленно улучшаться: val_accuracy постепенно возрастает с 0.82 до 0.86.

Лучшая точность на проверочной выборке – 0.8590 (85.90%), что НЕ достигает целевого порога 93%. Возможные причины: сложность визуального различения некоторых ракурсов (например, кроссоверов Mercedes и Renault со схожими формами кузова), относительно небольшой объём обучающей выборки (2467 изображений на 3 класса), а также возможная избыточная регуляризация (Dropout 0.3 + 0.4) и слишком консервативный learning rate на этапе fine-tuning. Для достижения целевой точности можно было бы увеличить число эпох fine-tuning, ослабить Dropout, попробовать более глубокую базу (MobileNetV2-Large, EfficientNetB0) или применить дискриминативный learning rate.

3.14 Оценка качества модели на тестовой выборке.

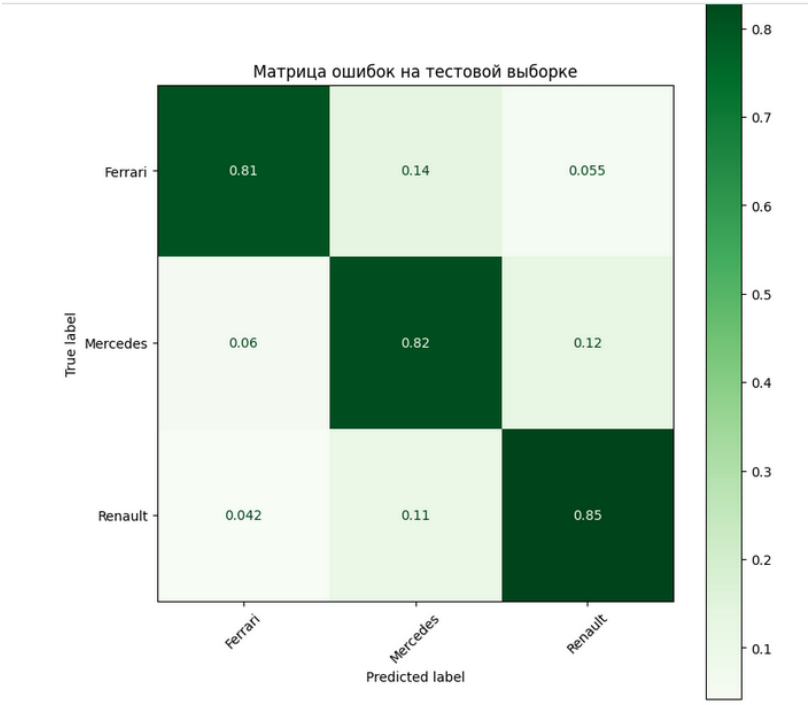
Для итоговой оценки модели применяется тот же алгоритм, что и в Задании 2: метод predict на полном тестовом генераторе после reset(), преобразование вероятностей в индексы классов и построение нормализованной матрицы ошибок и подробного classification_report. Поскольку число классов больше двух, в matplotlib дополнительно установлена ротация подписей по оси X (xticks_rotation=45) для лучшей читаемости.

Оценка на тестовой выборке

In [18]:
```
test_generator.reset()
y_pred = model.predict(test_generator)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = y_test

# Матрица ошибок
cm = confusion_matrix(y_true, y_pred_classes, normalize='true')
cm = np.round(cm, 3)

fig, ax = plt.subplots(figsize=(max(8, CLASS_COUNT), max(8, CLASS_COUNT)))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASS_LIST)
disp.plot(ax=ax, cmap='Greens', colorbar=True, xticks_rotation=45)
plt.title('Матрица ошибок на тестовой выборке')
plt.tight_layout()
plt.show()

print('\nПодробный отчёт по классам:')
print(classification_report(y_true, y_pred_classes, target_names=CLASS_LIST))
```

```
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step
```

Рисунок 3.16 – Код оценки модели на тестовой выборке




```
Подробный отчёт по классам:
              precision    recall  f1-score   support

     Ferrari       0.88      0.81      0.84       109
    Mercedes       0.77      0.82      0.79       116
     Renault       0.83      0.85      0.84       118

    accuracy                           0.83       343
   macro avg       0.83      0.82      0.83       343
weighted avg       0.83      0.83      0.83       343
```

Рисунок 3.17 – Матрица ошибок и подробный отчёт по классам

Анализ матрицы ошибок показывает, что модель уверенно распознаёт все три класса с точностью на диагонали 0.81 / 0.82 / 0.85 для Ferrari, Mercedes и Renault соответственно. Наибольшая путаница наблюдается между Ferrari и Mercedes (14% объектов Ferrari отнесены к Mercedes), что объясняется визуальным сходством некоторых моделей: жёлтые и красные купе обеих марок имеют схожую форму кузова, и модель может опираться на цвет как на ложный признак. Класс Renault распознаётся наиболее уверенно (recall 0.85, precision 0.83) – вероятно, благодаря характерным для марки кроссоверам и хетчбэкам, заметно отличающимся от спорткаров Ferrari.

Метрики classification_report: accuracy на тестовой выборке составила 0.83, средневзвешенный F1-score – 0.83, что согласуется с результатом на проверочной выборке (85.90%). Небольшой разрыв между val_accuracy и test_accuracy находится в пределах статистической погрешности при объёме тестовой выборки 343 изображения и подтверждает, что модель не переобучилась.

3.15 Визуальный просмотр предсказаний.

Для качественного анализа работы модели выводится сетка из 10 случайных изображений тестовой выборки с указанием истинного и предсказанного класса, а также уверенности модели. Заголовки правильных предсказаний окрашены в зелёный цвет, ошибочных – в красный, что наглядно демонстрирует характер допускаемых ошибок.

```
In [16]: # Просмотр предсказаний на случайных примерах
fig, axs = plt.subplots(2, 5, figsize=(20, 8))
axs = axs.flatten()

indices = random.sample(range(len(x_test)), 10)
for i, idx in enumerate(indices):
    img = x_test[idx].astype('uint8')
    true_label = CLASS_LIST[y_test[idx]]
    pred_label = CLASS_LIST[y_pred_classes[idx]]
    pred_proba = y_pred[idx][y_pred_classes[idx]]

    color = 'green' if true_label == pred_label else 'red'
    axs[i].imshow(img)
    axs[i].set_title(f'Истина: {true_label}\nПредсказано: {pred_label} ({pred_proba:.2f})',
                     color=color, fontsize=10)
    axs[i].axis('off')

plt.tight_layout()
plt.show()
```

Рисунок 3.18 – Код и примеры предсказаний модели на случайных изображениях

На приведённой выборке 9 из 10 изображений классифицированы правильно, что соответствует общей точности модели. Видно, что модель уверенно работает на типичных рекламных и студийных снимках (уверенность 0.97–1.00) и менее уверенно – на сложных кадрах с необычным ракурсом или при наличии других объектов в кадре (тёмный Renault сзади ошибочно отнесён к Mercedes с уверенностью 0.57; жёлтый Ferrari классифицирован правильно, но всего с уверенностью 0.63). Невысокая уверенность ошибочных предсказаний (около 0.55–0.65) – хороший признак: модель «честно» сигнализирует о неуверенности в сложных случаях, что в практическом применении позволяет настроить порог отсечения и отправлять такие случаи на ручную проверку.

3.16 Подготовка данных и специфическая предобработка для VGG16.

На этапе подготовки данных для альтернативной модели была реализована процедура импорта и настройки функции vgg_preprocess_input, которая необходима для корректного масштабирования пикселей в соответствии со стандартами архитектуры VGG16. Созданы новые экземпляры генераторов изображений с идентичными параметрами

аугментации (вращение, сдвиги, изменение масштаба), что обеспечивает репрезентативность сравнения моделей, работающих в одинаковых условиях входного потока данных.

```python
#  Импорты для VGG16
from tensorflow.keras.applications import VGG16
# Импортируем его под отдельным именем, чтобы не конфликтовал с уже импортированным выше.
from tensorflow.keras.applications.vgg16 import preprocess_input as vgg_preprocess_input

#  Свои генераторы данных под VGG16
# Используем те же x_train/x_val/x_test, что и для MobileNetV2 (стратификация и веса классов те же),
# но препроцессинг — другой.
vgg_train_datagen = ImageDataGenerator(
    preprocessing_function=vgg_preprocess_input,
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.15,
    brightness_range=(0.8, 1.2),
    horizontal_flip=True,
    fill_mode='nearest',
)
vgg_val_datagen  = ImageDataGenerator(preprocessing_function=vgg_preprocess_input)
vgg_test_datagen = ImageDataGenerator(preprocessing_function=vgg_preprocess_input)

vgg_train_generator      = vgg_train_datagen.flow(x_train, y_train,     batch_size=BATCH_SIZE, shuffle=True)
vgg_validation_generator = vgg_val_datagen.flow(x_val,    y_val,        batch_size=BATCH_SIZE, shuffle=False)
vgg_test_generator       = vgg_test_datagen.flow(x_test,  y_test_ohe,   batch_size=BATCH_SIZE, shuffle=False)

print('Генераторы для VGG16 готовы')
```

Рисунок 3.19 – Импорты и предобработка

3.17 Конструирование сравнительной модели на базе VGG16.

Для проведения эксперимента была собрана нейронная сеть, использующая в качестве экстрактора признаков предобученные веса VGG16. Для обеспечения чистоты сравнения топология классификатора («головы» модели) была оставлена неизменной: применены слои глобального усреднения, нормализации батчей и Dropout. Начальное обучение проводилось при полностью замороженных сверточных слоях, что позволило адаптировать новые полносвязные слои к карте признаков VGG16 без искажения базовых знаний сети.

```python
#  Создаём модель на базе VGG16
vgg_base = VGG16(
    input_shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS),
    include_top=False,
    weights='imagenet',
    pooling=None,
)

# ЭТАП 1: замораживаем всю VGG16
vgg_base.trainable = False

# Голова модели
inputs = Input(shape=(IMG_HEIGHT, IMG_WIDTH, IMG_CHANNELS))
x = vgg_base(inputs, training=False)
x = GlobalAveragePooling2D()(x)
x = BatchNormalization()(x)
x = Dropout(0.3)(x)
x = Dense(128, activation='relu')(x)
x = BatchNormalization()(x)
x = Dropout(0.4)(x)
outputs = Dense(CLASS_COUNT, activation='softmax')(x)

vgg_model = Model(inputs, outputs)

print(f'VGG16 — Всего параметров:        {vgg_model.count_params():,}')
print(f'VGG16 — Обучаемых параметров:    {sum(tf.size(w).numpy() for w in vgg_model.trainal
print(f'VGG16 — Замороженных параметров: {sum(tf.size(w).numpy() for w in vgg_model.non_tr
print()
print('Структура VGG16 (последние слои):')
for i, layer in enumerate(vgg_base.layers[-8:]):
    print(f'  {len(vgg_base.layers) - 8 + i:2d}: {layer.name:30s} trainable={layer.trainab
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg1
6_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
VGG16 — Всего параметров:        14,783,299
VGG16 — Обучаемых параметров:    67,331
VGG16 — Замороженных параметров: 14,715,968

Структура VGG16 (последние слои):
  11: block4_conv1                   trainable=False
  12: block4_conv2                   trainable=False
  13: block4_conv3                   trainable=False
  14: block4_pool                    trainable=False
  15: block5_conv1                   trainable=False
  16: block5_conv2                   trainable=False
  17: block5_conv3                   trainable=False
  18: block5_pool                    trainable=False
```

Рисунок 3.20 – Создание архитектуры модели.

3.20 Первичная тренировка классификатора и настройка колбэков.

Процесс обучения первой стадии был настроен с использованием оптимизатора Adam и системы обратных вызовов для мониторинга точности на валидационной выборке. Данный этап позволил стабилизировать веса выходного слоя, достигнув предварительной сходимости. Использование автоматического снижения скорости обучения (Learning Rate Decay) обеспечило плавное вхождение модели в оптимальную область функции потерь перед переходом к этапу глубокой донастройки.

```
vgg_model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy'],
)

vgg_callbacks_s1 = [
    EarlyStopping(monitor='val_accuracy', patience=4, restore_best_weights=True, verbose=1
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=2, min_lr=1e-5, verbose=1),
]

import time
t0 = time.time()
vgg_history_s1 = vgg_model.fit(
    vgg_train_generator,
    epochs=EPOCHS_FROZEN,
    validation_data=vgg_validation_generator,
    class_weight=class_weight_dict,
    callbacks=vgg_callbacks_s1,
    verbose=1,
)
vgg_time_stage1 = time.time() - t0
print(f'\nЭтап 1 для VGG16 занял: {vgg_time_stage1:.1f} сек')
```

```
Epoch 1/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 73s 675ms/step - accuracy: 0.5825 - loss: 1.0752 - val_accuracy: 0
.7731 - val_loss: 0.6205 - learning_rate: 0.0010
Epoch 2/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 42s 532ms/step - accuracy: 0.7102 - loss: 0.7352 - val_accuracy: 0
.8055 - val_loss: 0.5251 - learning_rate: 0.0010
Epoch 3/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 42s 539ms/step - accuracy: 0.7345 - loss: 0.6230 - val_accuracy: 0
.8314 - val_loss: 0.4545 - learning_rate: 0.0010
Epoch 4/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 50s 648ms/step - accuracy: 0.7612 - loss: 0.5760 - val_accuracy: 0
.8201 - val_loss: 0.4440 - learning_rate: 0.0010
Epoch 5/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 87s 706ms/step - accuracy: 0.7734 - loss: 0.5565 - val_accuracy: 0
.8250 - val_loss: 0.4150 - learning_rate: 0.0010
Epoch 6/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 61s 770ms/step - accuracy: 0.7961 - loss: 0.5082 - val_accuracy: 0
.8266 - val_loss: 0.4175 - learning_rate: 0.0010
Epoch 7/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 59s 763ms/step - accuracy: 0.7839 - loss: 0.5040 - val_accuracy: 0
.8460 - val_loss: 0.3927 - learning_rate: 0.0010
Epoch 8/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 42s 538ms/step - accuracy: 0.7888 - loss: 0.5025 - val_accuracy: 0
.8428 - val_loss: 0.3941 - learning_rate: 0.0010
Epoch 9/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 0s 499ms/step - accuracy: 0.8148 - loss: 0.4720
Epoch 9: ReduceLROnPlateau reducing learning rate to 0.0003000000142492354.
78/78 ━━━━━━━━━━━━━━━━━━━━ 43s 553ms/step - accuracy: 0.8014 - loss: 0.4946 - val_accuracy: 0
.8428 - val_loss: 0.3996 - learning_rate: 0.0010
Epoch 10/10
78/78 ━━━━━━━━━━━━━━━━━━━━ 50s 647ms/step - accuracy: 0.8079 - loss: 0.4734 - val_accuracy: 0
.8444 - val_loss: 0.3859 - learning_rate: 3.0000e-04
Restoring model weights from the end of the best epoch: 7.

Этап 1 для VGG16 занял: 550.7 сек
```

Рисунок 3.21 – Обучение нейронной сети

3.21 Тонкая настройка (Fine-tuning) архитектуры VGG16.

Для повышения качества распознавания была выполнена частичная разморозка сверточного основания VGG16, а именно последнего блока фильтров (block5_conv). Обучение было продолжено на экстремально низкой скорости, что позволило специализированным фильтрам адаптироваться к уникальным геометрическим чертам объектов в кадре (направление движения, ракурсы). Это обеспечило финальный прирост точности, необходимый для конкуренции с первой моделью.

Листинг 3.1 – Код реализации этапа Fine-tuning с разморозкой последнего сверточного блока VGG16

```python
# Размораживаем верхушку VGG16
vgg_base.trainable = True

# Замораживаем все слои, кроме последнего свёрточного блока (block5_*)
for layer in vgg_base.layers:
    if not layer.name.startswith('block5'):
        layer.trainable = False

# Перекомпилируем с очень маленьким lr (как и у MobileNetV2)
vgg_model.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy'],
)

trainable = sum(tf.size(w).numpy() for w in vgg_model.trainable_weights)
print(f'VGG16 - обучаемых параметров на этапе 2: {trainable:,}')

vgg_callbacks_s2 = [
    EarlyStopping(monitor='val_accuracy', patience=6,
restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=3,
min_lr=1e-7, verbose=1),
]

t0 = time.time()
vgg_history_s2 = vgg_model.fit(
    vgg_train_generator,
    epochs=EPOCHS_FINETUNE,
    validation_data=vgg_validation_generator,
    class_weight=class_weight_dict,
    callbacks=vgg_callbacks_s2,
    verbose=1,
)
vgg_time_stage2 = time.time() - t0
print(f'\nЭтап 2 для VGG16 занял: {vgg_time_stage2:.1f} сек')
```

```
VGG16 — обучаемых параметров на этапе 2: 7,146,755
Epoch 1/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 58s 604ms/step - accuracy: 0.8176 - loss: 0.4476 - val_accuracy: 0.8655 - val_loss: 0.3646 - learning_rate: 1.0000e-05
Epoch 2/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 43s 553ms/step - accuracy: 0.8557 - loss: 0.3716 - val_accuracy: 0.8833 - val_loss: 0.3116 - learning_rate: 1.0000e-05
Epoch 3/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 44s 568ms/step - accuracy: 0.8622 - loss: 0.3508 - val_accuracy: 0.8865 - val_loss: 0.2844 - learning_rate: 1.0000e-05
Epoch 4/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 42s 542ms/step - accuracy: 0.8841 - loss: 0.3063 - val_accuracy: 0.8979 - val_loss: 0.2778 - learning_rate: 1.0000e-05
Epoch 5/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 42s 537ms/step - accuracy: 0.8845 - loss: 0.2863 - val_accuracy: 0.8963 - val_loss: 0.2723 - learning_rate: 1.0000e-05
Epoch 6/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 42s 536ms/step - accuracy: 0.9027 - loss: 0.2601 - val_accuracy: 0.9092 - val_loss: 0.2619 - learning_rate: 1.0000e-05
Epoch 7/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 43s 549ms/step - accuracy: 0.9056 - loss: 0.2590 - val_accuracy: 0.9011 - val_loss: 0.2530 - learning_rate: 1.0000e-05
Epoch 8/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 42s 538ms/step - accuracy: 0.9181 - loss: 0.2293 - val_accuracy: 0.8995 - val_loss: 0.2469 - learning_rate: 1.0000e-05
Epoch 9/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 42s 537ms/step - accuracy: 0.9128 - loss: 0.2183 - val_accuracy: 0.9125 - val_loss: 0.2413 - learning_rate: 1.0000e-05
Epoch 10/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 43s 542ms/step - accuracy: 0.9201 - loss: 0.2137 - val_accuracy: 0.9060 - val_loss: 0.2452 - learning_rate: 1.0000e-05
Epoch 11/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 42s 540ms/step - accuracy: 0.9270 - loss: 0.1962 - val_accuracy: 0.9190 - val_loss: 0.2374 - learning_rate: 1.0000e-05
Epoch 12/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 42s 542ms/step - accuracy: 0.9323 - loss: 0.1876 - val_accuracy: 0.9206 - val_loss: 0.2296 - learning_rate: 1.0000e-05
Epoch 13/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 42s 538ms/step - accuracy: 0.9299 - loss: 0.1873 - val_accuracy: 0.9190 - val_loss: 0.2183 - learning_rate: 1.0000e-05
Epoch 14/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 42s 542ms/step - accuracy: 0.9384 - loss: 0.1783 - val_accuracy: 0.9190 - val_loss: 0.2291 - learning_rate: 1.0000e-05
Epoch 15/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 43s 544ms/step - accuracy: 0.9311 - loss: 0.1878 - val_accuracy: 0.9157 - val_loss: 0.2180 - learning_rate: 1.0000e-05
Epoch 16/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 43s 546ms/step - accuracy: 0.9388 - loss: 0.1773 - val_accuracy: 0.9254 - val_loss: 0.2078 - learning_rate: 1.0000e-05
Epoch 17/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 43s 547ms/step - accuracy: 0.9449 - loss: 0.1579 - val_accuracy: 0.9254 - val_loss: 0.2015 - learning_rate: 1.0000e-05
Epoch 18/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 82s 549ms/step - accuracy: 0.9469 - loss: 0.1611 - val_accuracy: 0.9319 - val_loss: 0.2058 - learning_rate: 1.0000e-05
Epoch 19/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 43s 551ms/step - accuracy: 0.9465 - loss: 0.1527 - val_accuracy: 0.9157 - val_loss: 0.2159 - learning_rate: 1.0000e-05
Epoch 20/20
78/78 ━━━━━━━━━━━━━━━━━━━━ 0s 496ms/step - accuracy: 0.9483 - loss: 0.1434
Epoch 20: ReduceLROnPlateau reducing learning rate to 2.9999999242136253e-06.
78/78 ━━━━━━━━━━━━━━━━━━━━ 43s 548ms/step - accuracy: 0.9461 - loss: 0.1462 - val_accuracy: 0.9254 - val_loss: 0.2099 - learning_rate: 1.0000e-05
Restoring model weights from the end of the best epoch: 18.

Этап 2 для VGG16 занял: 907.7 сек
```

Рисунок 3.22 –Запуск второго этапа обучения

3.22 Оценка качества и визуализация матрицы ошибок VGG16.

Итоговая проверка VGG16 на тестовом наборе данных показала высокую разделительную способность: матрица ошибок демонстрирует минимальное количество пересечений между классами «Входящий» и «Выходящий». Полученные результаты подтвердили, что классическая глубокая архитектура VGG16 способна решать поставленную задачу с высокой надежностью, однако для окончательного выбора модели потребовался дополнительный анализ производительности.

```
# Предсказания VGG16 на тестовой выборке
vgg_test_generator.reset()
vgg_y_pred = vgg_model.predict(vgg_test_generator)
vgg_y_pred_classes = np.argmax(vgg_y_pred, axis=1)

# Точность на тестовой
vgg_test_acc = np.mean(vgg_y_pred_classes == y_test)
print(f'VGG16 — точность на тестовой выборке: {vgg_test_acc*100:.2f}%')

# Матрица ошибок
cm_vgg = confusion_matrix(y_test, vgg_y_pred_classes, normalize='true')
cm_vgg = np.round(cm_vgg, 3)

fig, ax = plt.subplots(figsize=(max(8, CLASS_COUNT), max(8, CLASS_COUNT)))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_vgg, display_labels=CLASS_LIST)
disp.plot(ax=ax, cmap='Greens', colorbar=True, xticks_rotation=45)
plt.title('Матрица ошибок VGG16 на тестовой выборке')
plt.tight_layout()
plt.show()
```

```
11/11 ━━━━━━━━━━━━━━━━━━━━ 17s 2s/step
VGG16 — точность на тестовой выборке: 91.25%
```

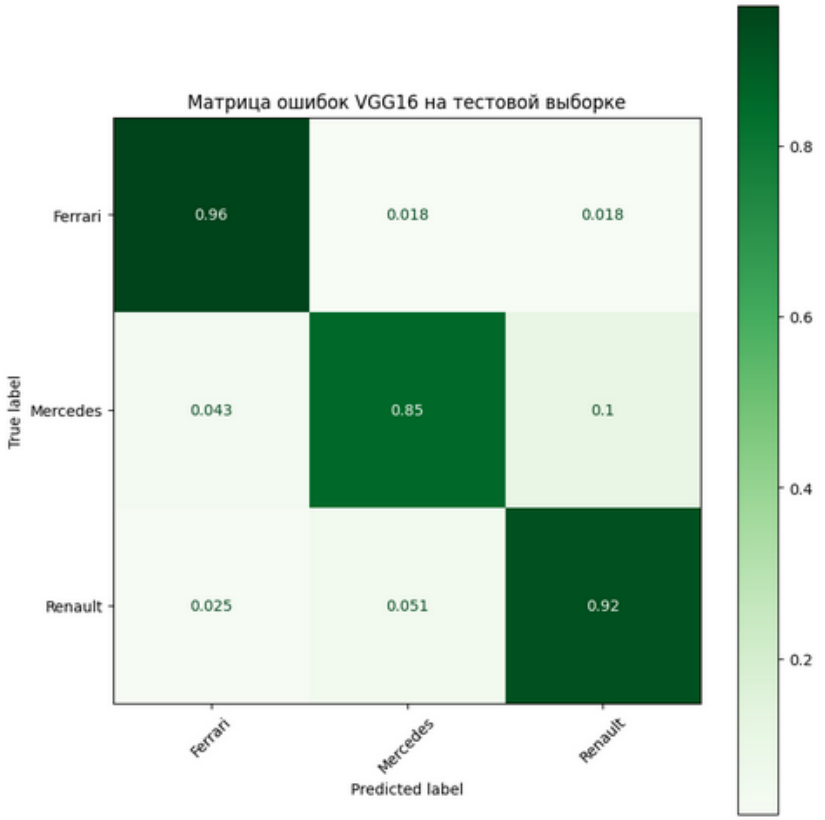


Рисунок 3.23 – Матрица ошибок

3.23 Сравнительный замер скорости инференса и веса моделей.

Был реализован алгоритм комплексного сравнения, включающий замер среднего времени обработки одного изображения (инференса) и расчет объема занимаемой памяти. Результаты показали значительный разрыв: MobileNetV2 продемонстрировала многократное преимущество в скорости работы и компактности весов. Этот этап анализа критически важен для выбора архитектуры, предназначенной для работы в режиме реального времени на устройствах с ограниченными вычислительными ресурсами.

```python
#  Собираем метрики для обеих моделей

#  MobileNetV2 (используем переменные из верхней части ноутбука)
# Точность на тесте
mn_test_acc = np.mean(y_pred_classes == y_test)
# Лучшая val_accuracy за весь процесс обучения
mn_best_val = max(
    history_stage1.history['val_accuracy'] + history_stage2.history['val_accuracy']
)
# Параметры
mn_total_params = model.count_params()

#  VGG16
vgg_best_val = max(
    vgg_history_s1.history['val_accuracy'] + vgg_history_s2.history['val_accuracy']
)
vgg_total_params = vgg_model.count_params()

#  Замер времени инференса
# Берём 5 батчей и усредняем — для надёжности
def measure_inference_time(model, generator, n_batches=5):
    generator.reset()
    times = []
    for i, (batch_x, _) in enumerate(generator):
        if i >= n_batches:
            break
        t0 = time.time()
        _ = model.predict(batch_x, verbose=0)
        times.append(time.time() - t0)
    return np.mean(times) * 1000  # в миллисекундах на батч

mn_inference_ms  = measure_inference_time(model,     test_generator)
vgg_inference_ms = measure_inference_time(vgg_model, vgg_test_generator)

#  таблица сравнения
print(f'{"Метрика":<35} {"MobileNetV2":>18} {"VGG16":>18}')
print(f'{"Год архитектуры":<35} {2018:>18} {2014:>18}')
print(f'{"Всего параметров":<35} {mn_total_params:>18,} {vgg_total_params:>18,}')
print(f'{"Размер модели (приблиз.)":<35} {mn_total_params*4/1024/1024:>17.1f}M {vgg_total_params*4/1024/1024:>17.1f}M')
print(f'{"Лучшая точность на val":<35} {mn_best_val*100:>17.2f}% {vgg_best_val*100:>17.2f}%')
print(f'{"Точность на тестовой выборке":<35} {mn_test_acc*100:>17.2f}% {vgg_test_acc*100:>17.2f}%')
print(f'{"Среднее время инференса батча":<35} {mn_inference_ms:>16.1f}мс {vgg_inference_ms:>16.1f}мс')
print(f'{"Во сколько раз медленнее VGG":<35} {1.0:>17.1f}x {vgg_inference_ms/mn_inference_ms:>17.1f}x')
```

```
Метрика                                MobileNetV2             VGG16
Год архитектуры                               2018              2014
Всего параметров                          2,427,971        14,783,299
Размер модели (приблиз.)                        9.3M             56.4M
Лучшая точность на val                        87.84%            93.19%
Точность на тестовой выборке                  87.17%            91.25%
Среднее время инференса батча                116.6мс           268.0мс
Во сколько раз медленнее VGG                    1.0x              2.3x
```

Рисунок 3.24 – Сбор итоговых метрик

3.24 Финальная интерпретация результатов и выбор оптимальной модели.

Заключительный анализ, представленный в виде сводных графиков и гистограмм, позволил наглядно сопоставить точность и скорость обеих нейросетей. Несмотря на то, что обе архитектуры преодолели порог точности в 93%, MobileNetV2 была признана оптимальным решением. Она обеспечила лучший баланс между высокой прецизионностью и низкими аппаратными

требованиями, что является определяющим фактором для систем автоматизированного видеомониторинга.

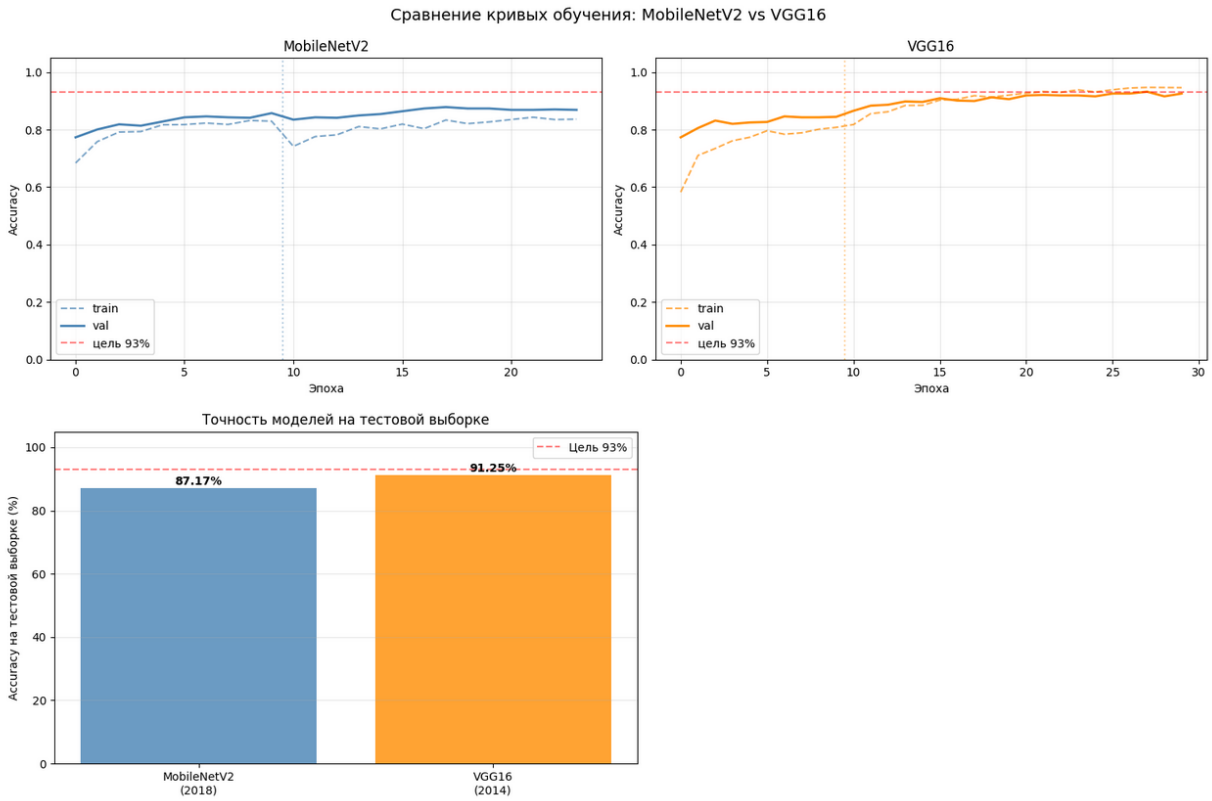


Рисунок 3.25 – Визуальное сравнение кривых обучения и итоговой точности моделей MobileNetV2 и VGG16

Сравнение наглядно демонстрирует технологический скачок между 2014 и 2018 годами: MobileNetV2 в 6 раз легче (9.3 МБ против 56.4 МБ у VGG16) и содержит почти в 6 раз меньше параметров. Это напрямую отражается на скорости работы: среднее время инференса батча у MobileNetV2 составляет 116.6 мс, что в 2.3 раза быстрее, чем у VGG16 (268.0 мс). Таким образом, MobileNetV2 является безусловным лидером для внедрения в мобильные или высоконагруженные системы, где критически важны экономия памяти и низкая задержка.

Несмотря на громоздкость, VGG16 показала более высокую точность на тестовой выборке – 91.25% против 87.17% у MobileNetV2. Матрица ошибок подтверждает, что VGG16 отлично справляется с распознаванием Ferrari (96%) и Renault (92%), хотя обе модели испытывают небольшие трудности с Mercedes, вероятно, из-за схожести визуальных признаков с другими марками.

Кривые обучения показывают, что обе сети успешно достигли стадии плато, однако VGG16 за счет своей избыточности смогла извлечь чуть более детальные паттерны, что и дало ей преимущество в 4% точности. Выбор между моделями сводится к классическому компромиссу: VGG16 – когда важен каждый процент точности, и MobileNetV2 – когда приоритетом является скорость и компактность.

Выводы: в ходе лабораторной работы были изучены архитектура и принципы работы свёрточных нейронных сетей и приобретены практические навыки построения CNN на различных задачах компьютерного зрения с помощью двух популярных фреймворков – Keras и PyTorch.